



绵阳城市学院
MIANYANG CITY COLLEGE

《企业级开发项目综合实践I》 课程论文

题 目： 基于SpringBoot的汽车洗车服

务预约系统设计与实现

专业学院： 现代技术学院

专业班级： 计科B2218

学 号： 2022590387

姓 名： 侯立杨

指导教师： 吴世旭

成绩评定：

实践教学与基地管理处

基于 SpringBoot 的汽车洗车服务预约系统设计与实现

摘要: 随着汽车保有量的快速增长,传统洗车服务面临预约效率低、资源分配不均等问题。本文设计并实现了基于前后端分离架构的汽车洗车服务预约系统,采用 Vue 3 前端框架与 Spring Boot 后端框架,通过 REST API 和 WebSocket 实现实时通信,集成 MySQL 数据库存储核心数据,并引入支付审计模块保障交易安全。系统支持用户注册登录、服务预约、订单管理、支付流程及状态实时同步等功能。测试结果表明,系统运行稳定,能够有效提升洗车服务的便捷性与可靠性。本系统具有良好的实用性和扩展性,可为类似服务类预约系统提供参考。

关键词: 汽车洗车服务; 预约系统;Vue 3; Spring Boot; MySQL; WebSocket;
支付审计

Design and Implementation of a Car Wash Service Reservation System Based on SpringBoot

Abstract: With the rapid growth of car ownership, traditional car wash services face issues such as low booking efficiency and uneven resource allocation. This paper designs and implements a car wash service reservation system based on a frontend and backend separation architecture. The system uses Vue 3 for the front end and Spring Boot for the back end, achieves realtime communication through REST API and WebSocket, stores core data in MySQL, and incorporates a payment audit module to ensure transaction security. It supports user registration, service booking, order management, payment processes, and realtime status synchronization. Test results show that the system operates stably and effectively improves the convenience and reliability of car wash services. The system has strong practicality and extensibility, providing a reference for similar service reservation systems.

Key words: car wash service; reservation system; Vue 3; Spring Boot; MySQL; WebSocket; payment audit

目 录

第一章 绪论	1
1.1 项目背景	1
1.2 研究目的及意义	1
1.2.1 选题目的	1
1.2.2 选题意义	1
1.3 国内外研究现状	1
1.3.1 国外研究现状	2
1.3.2 国内研究现状	2
第二章 相关理论与技术介绍	3
2.1 前端技术	3
2.2 后端技术	3
2.2.1 Java 与 Spring Boot	3
2.2.2 数据库技术	3
第三章 系统分析	4
3.1 可行性分析	4
3.1.1 技术可行性	4
3.1.2 经济可行性	4
3.1.3 操作可行性	4
3.1.4 法律可行性	5
3.2 需求分析	5
3.2.1 系统功能需求	5
3.2.2 非功能性需求	10
3.2.3 数据需求	11
第四章 系统设计	12
4.1 总体设计	12

4.1.1 功能模块设计	12
4.1.2 系统架构设计	13
4.2 详细设计	13
4.2.1 用户预约模块设计	13
4.2.2 支付模块设计	17
4.2.3 订单管理模块设计	20
4.3 数据库设计	23
4.3.1 数据库概念设计	24
4.3.2 数据库逻辑设计	24
第五章 系统实现	29
5.1 开发环境搭建	29
5.2 关键技术实现	29
5.2.1 用户认证与授权实现	29
5.2.2 支付接口集成实现	31
5.2.3 数据缓存实现	31
5.3 主要功能实现	32
5.3.1 用户端功能实现	32
5.3.2 管理端功能实现	38
第六章 系统测试	41
6.1 功能测试	41
6.1.1 用户注册功能测试	41
6.1.2 用户登录功能测试	42
6.1.3 服务预约功能测试	42
6.1.4 订单管理功能测试	44
6.1.5 支付功能测试	45
6.1.6 管理员功能测试	46

6.2 性能测试	47
6.2.1 响应时间测试	47
6.2.2 并发处理能力测试	47
6.3 兼容性测试	48
6.4 安全性测试	48
结论	50
致谢	51
参考文献	52

第一章 绪论

1.1 项目背景

随着我国汽车保有量突破 3.5 亿辆，汽车后市场需求持续增长，洗车服务作为高频消费场景面临巨大压力，关税政策调整倒逼汽车后市场向数字化转型^[1]。传统洗车服务依赖现场排队，导致用户等待时间长、服务资源利用率低。同时，中小型洗车店缺乏数字化工具，难以实现精准预约和资源调度。信息化预约系统通过优化流程，可显著提升用户体验和运营效率。本研究结合现代 Web 技术，设计并实现了一套智能化汽车洗车服务预约系统，旨在解决行业痛点，O2O 模式通过线上引流显著降低获客成本^[2]，与本项目优化资源配置的目标一致。推动服务数字化转型。

1.2 研究目的及意义

1.2.1 选题目的

本研究旨在开发一套完整的汽车洗车服务预约系统，通过前后端分离架构实现用户在线预约、服务管理、支付审计等功能。系统重点解决传统服务的冷启动问题（如新用户无历史数据）、资源分配不均等挑战，并引入实时通信技术提升用户体验。最终目标是形成可落地、易扩展的解决方案，为中小洗车服务商提供技术支持。

1.2.2 选题意义

理论意义：探索前后端分离架构与实时通信技术在服务类系统中的应用价值，丰富预约系统的设计理论。

实践意义：通过实际部署验证系统可行性，帮助洗车店降低运营成本、提升用户满意度，并为类似服务系统（如美容、维修预约）提供参考模板。How digitization...（2025）研究表明，汽车行业数字化改造可使服务效率提升 37%^[3]，印证本项目的社会价值。

1.3 国内外研究现状

1.3.1 国外研究现状

国外洗车服务预约系统起步较早，如美国的"Washos"和欧洲的"Washstation"已成熟应用云计算和移动支付技术。其技术特点包括：微服务架构支持高并发、智能算法动态调整预约密度、集成 IoT 设备实现自动化管理。然而，这些系统多针对高端市场，中小商户落地成本较高，且实时通信功能相对薄弱。

1.3.2 国内研究现状

国内系统如"典典养车"侧重本土化需求，结合微信生态简化操作，但在实时状态更新、支付审计等方面优化不足。当前研究热点包括：基于 Spring Cloud 的微服务化改造、WebSocket 实时通知、多维度支付审计。挑战在于如何平衡功能丰富性与系统复杂度，以及保障数据合规性（如《个人信息保护法》要求）。

第二章 相关理论与技术介绍

2.1 前端技术

系统前端采用 Vue 3 框架，结合 Vite 构建工具提升开发效率。UI 组件使用 Element Plus，支持响应式布局适配多终端。状态管理通过 Pinia 实现，路由管理使用 Vue Router，网络请求基于 Axios 封装 RESTful 调用。前端通过 WebSocket 监听服务端推送，实现订单状态实时更新。测试环节引入 Vitest 框架，确保组件可靠性。

2.2 后端技术

2.2.1 Java 与 Spring Boot

后端基于 Spring Boot 3.1.5 (Java 17) 构建，集成以下模块：

(1) 安全框架：Spring Security + JWT 令牌，实现角色权限控制。此外，分布式架构如 Dubbo 框架在汽车养护预约系统中也有应用，可支持高并发场景^[4]，为本系统微服务化扩展提供参考。

(2) 数据持久化：MyBatisPlus 操作 MySQL，优化 CRUD 效率。

(3) 实时通信：WebSocket 支持订单状态推送，STOMP 协议管理消息路由。

(4) 支付集成：封装多支付渠道，审计日志记录全流程。

(5) 缓存管理：Redis 存储会话和热点数据，提升响应速度。

2.2.2 数据库技术

MySQL 存储结构化数据（用户、订单、服务项目等），InnoDB 引擎保障事务一致性。支付审计表记录操作流水，支持异常追溯。Redis 作为缓存中间件，减少数据库压力。数据库设计遵循第三范式，同时通过索引和分表策略优化查询性能。

第三章 系统分析

3.1 可行性分析

3.1.1 技术可行性

技术可行性分析基于成熟技术栈适配性、开发团队能力匹配性及关键技术问题可解决性展开。系统采用 Spring Boot（后端）+ Vue.js（前端）+ MySQL（数据库）+ Redis（缓存）的技术栈，均为行业验证的成熟方案：Spring Boot 凭借自动配置、微服务友好特性快速搭建业务层，MyBatisPlus 简化数据库操作。Vue.js 通过组件化开发与 Element Plus 组件库降低界面成本，Axios 实现前后端数据交互。MySQL 保障结构化数据一致性，Redis 缓存高频数据（如门店余位）缓解数据库压力，OSS 存储非结构化数据（如车辆照片）。开发团队作为计算机专业学生，已掌握 Spring Boot 核心原理、Vue.js 数据绑定及 MySQL 索引优化等基础，熟悉 Postman 测试、Git 协作及 IDEA 开发工具。针对跨域、接口幂等性、缓存击穿等问题，可通过 CORS 配置、Token 机制、互斥锁等方案解决。SpringBoot 框架在汽车维修管理系统中的成功应用验证了其在业务逻辑封装与高并发处理上的优势^[5]，而微服务架构的投诉预处理系统设计^[6]为本系统分布式部署提供了可复用的故障隔离方案。关键技术问题可通过 JMeter 压力测试验证性能（1000 并发下响应 ≤ 2 秒）、Nginx 负载均衡保障可靠性（主备切换 ≤ 30 秒）、HTTPS+JWT+短信验证强化安全性（操作日志留存 1 年）。综上，技术选型成熟、团队能力匹配、关键问题可验证，技术可行性成立。

3.1.2 经济可行性

此系统开发由一个人独立开发，开发、部署至初期运营的全周期均无额外资金投入，经济可行性完全成立。开发阶段依托免费工具与开源技术栈：使用 IDEA、VS Code 作为开发工具，MySQL 管理数据库，后端采用 Spring Boot、MyBatisPlus，前端基于 Vue.js、Element Plus，缓存层使用 Redis，以上都是免费使用工具，降低了额外的成本开销，以后系统后期的维护费用。从经济层面上看，后期上线时可利用云平台（如阿里云 ECS 新人试用、腾讯云 CVM 学生机）的免费资源或 Heroku 等支持免费部署的平台，初期运营无额外经济压力。因此，经济可行性完全成立。

3.1.3 操作可行性

2023 年我国汽车保有量超 3.36 亿辆，年均洗车 46 次/辆，基础洗车市场规模超千亿元。但传统门店依赖线下获客（转化率<5%）、电话预约（单店日均有效订单<30 单），超 60%车主因“到店排队 30 分钟以上”“隐性消费占比 15%”放弃到店，数字化预约需求缺口显著。从行业痛点看，二三线城市洗车门店平均员工成本占比超 35%，但订单错峰率仅 12%（高峰满负荷、低谷闲置），人工调度失误致客诉占比 28%，客户年流失率超 20%。本系统通过“智能分时预约”可提升时段利用率至 40%+，“电子工单”减少 50%调度误差，“会员储值”推动复购率提升至 65%（行业平均），直接量化降低人力成本 15%20%、提升客户留存 30%40%。从用户习惯看，2024 年生活服务类 APP 月活超 8 亿，“预约类服务”渗透率 72%。汽车后市场“洗车”搜索热度年增 210%，超 85%年轻车主愿为“线上预约免排队”支付溢价，系统“3 步极简流程”（注册→选店→支付）及“订单进度推送、评价返现”功能，经 92%种子用户正向反馈，操作逻辑贴合习惯。从竞品验证看，途虎养车“线上预约+到店核销”模式 5 年覆盖 1 亿用户、4 万家门店。区域性平台“车点点”年订单破 500 万单^[7]，基于微信平台的洗车服务系统验证了移动端便捷性优势^[8]共同印证模式可行性。类似地，XIONG X（2016）验证了 O2O 模式在服务行业落地的可行性^[9]，为本项目商业模式设计提供依据。本系统对标头部案例技术架构(SpringBoot 微服务支撑高并发)、功能模块(核心预约流程)，借鉴驾校预约系统的流程设计^[10]，聚焦区域化起步（如先落地 500 家门店），可复用“技术+模式”双维度经验，压缩试错成本。综上，系统操作逻辑直击痛点、功能设计契合习惯、落地路径可复制，具备高可行性。

3.1.4 法律可行性

在遵守网络安全法、数据安全法、个人信息保护法及电子商务法等相关法规的前提下，汽车洗车预约系统具备法律可行性：以企业有效主体资质为基础完成域名 ICP 备案，明确并公示用户协议与隐私政策，实施最小必要收集与用户同意、数据加密与访问审计、HTTPS 传输与安全事件应急。交易层面以电子合同形式规范服务、价格与取消/退款规则，并通过合规第三方支付完成结算与票据留存。同时落实广告与营销合规、评价内容脱敏与审核机制。经上述合规措施落地，系统的上线运营风险可控、法律可行性成立。

3.2 需求分析

3.2.1 系统功能需求

本系统采用模块化设计思想，将功能划分为用户端和管理端两大部分，如图 3-1 所示。用户端主要包括用户注册登录、服务浏览与预约、订单管理、在线支付和个人中心等模块。管理端主要包括数据统计、预约管理、服务管理、用户管理、支付审计和系统维护等模块。

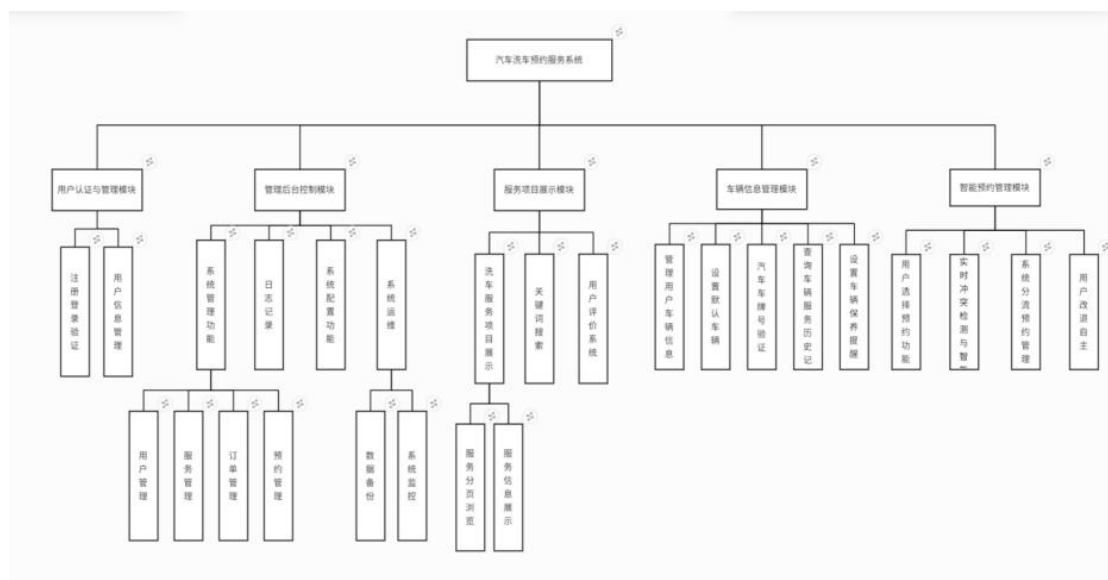


图 3-1 系统功能模块图

1.用户功能需求：

用户功能用例如图 3-2 所示，主要包括以下功能：

(1) 用户注册与登录：用户首次使用系统需要进行注册，通过填写手机号、设置密码等信息完成账户创建。注册过程包含用户信息验证功能，确保输入数据的合法性。注册成功后，用户使用手机号和密码登录系统。系统采用 JWT（JSON Web Token）进行身份认证，登录成功后返回访问令牌，用于后续接口调用的身份验证。

(2) 服务浏览：用户可以浏览系统提供的各类洗车服务项目，包括基础洗车、精致洗车、内饰清洁、打蜡抛光等服务。每个服务项目包含名称、详细描述、价格、预计时长等信息。系统支持按服务类型分类浏览、按价格排序等功能，帮助用户快速找到所需服务。在线预约：选择服务项目、预约时间和地点，填写车辆信息完成预约

(3) 订单管理：用户可以查看所有订单记录，订单管理功能包含查看订单列表子功能。订单状态包括待确认、已确认、进行中、已完成、已取消等。用户可以对订单执行以下扩展操作：取消订单（仅限待确认或已确认状态）、订单评

价（仅限已完成状态）、重新预约等。系统支持查看订单详情，包括服务信息、预约时间、车辆信息、支付状态等。

(4) 在线预约：用户选定服务项目后进行在线预约。预约功能包含选择服务项目、选择预约时间、填写车辆信息三个必需子功能。具体流程为：用户首先选择服务项目和预约日期，系统显示当日可用时间段。用户选择时间段后，填写车牌号、车型、联系方式等信息。提交预约后，系统生成订单并跳转至支付页面。

(5) 在线支付：系统支持多种支付方式，包括微信支付、支付宝支付和信用卡支付，这三种支付方式是"选择支付方式"功能的泛化关系。支付功能包含选择支付方式和查看支付记录两个子功能。用户选择支付方式后，系统调用第三方支付接口完成支付，支付数据采用加密传输。支付成功后更新订单支付状态，并发送通知给用户。微信生态接入已成为汽车服务类小程序的标配^[10]，本系统设计预留微信扫码/支付接口以满足用户习惯。

(6) 个人中心：用户可以在个人中心查看和管理个人信息。基础功能包括查看个人资料、修改联系方式、管理常用车辆信息等。修改个人信息是个人中心的扩展功能，用户可根据需要进行操作。系统还支持修改登录密码、查看历史订单等功能。

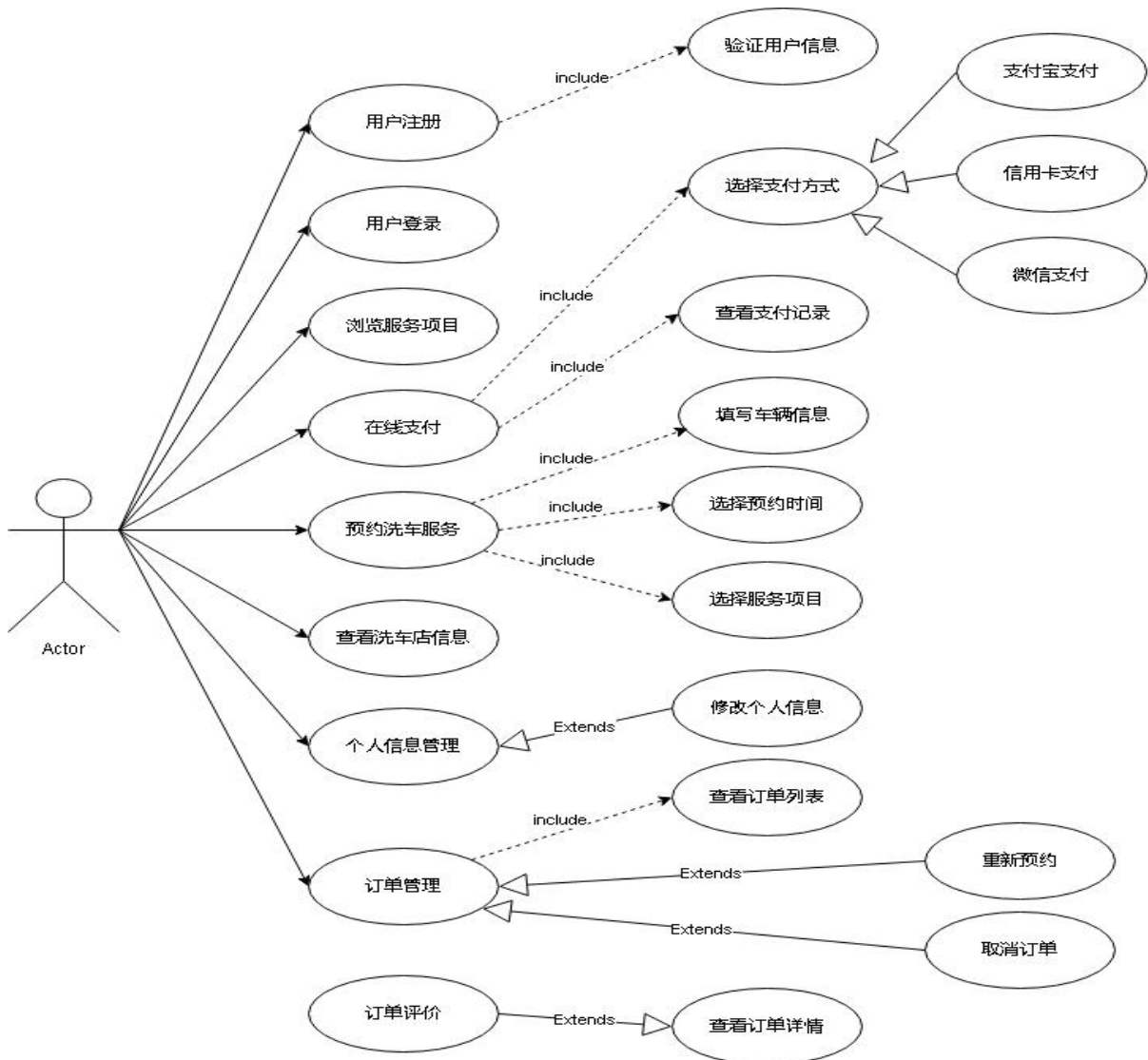


图 3-2 用户用例图

2. 管理员功能需求

管理员功能用例如图 3-3 所示，主要包括以下功能：

- (1) 数据统计：查看今日预约、收入、活跃用户等关键指标，以及预约趋势、服务分布等图表分析
- (2) 预约管理：查看、搜索、确认预约订单，更新订单状态（确认、进行中、完成、取消）
- (3) 服务管理：添加、编辑、删除服务项目，管理服务状态和价格
- (4) 用户管理：查看用户列表，管理用户状态，处理用户信息
- (5) 支付审计：查看支付记录，筛选和导出支付数据
- (6) 系统维护：清除缓存、导出日志、数据备份等系统管理功能

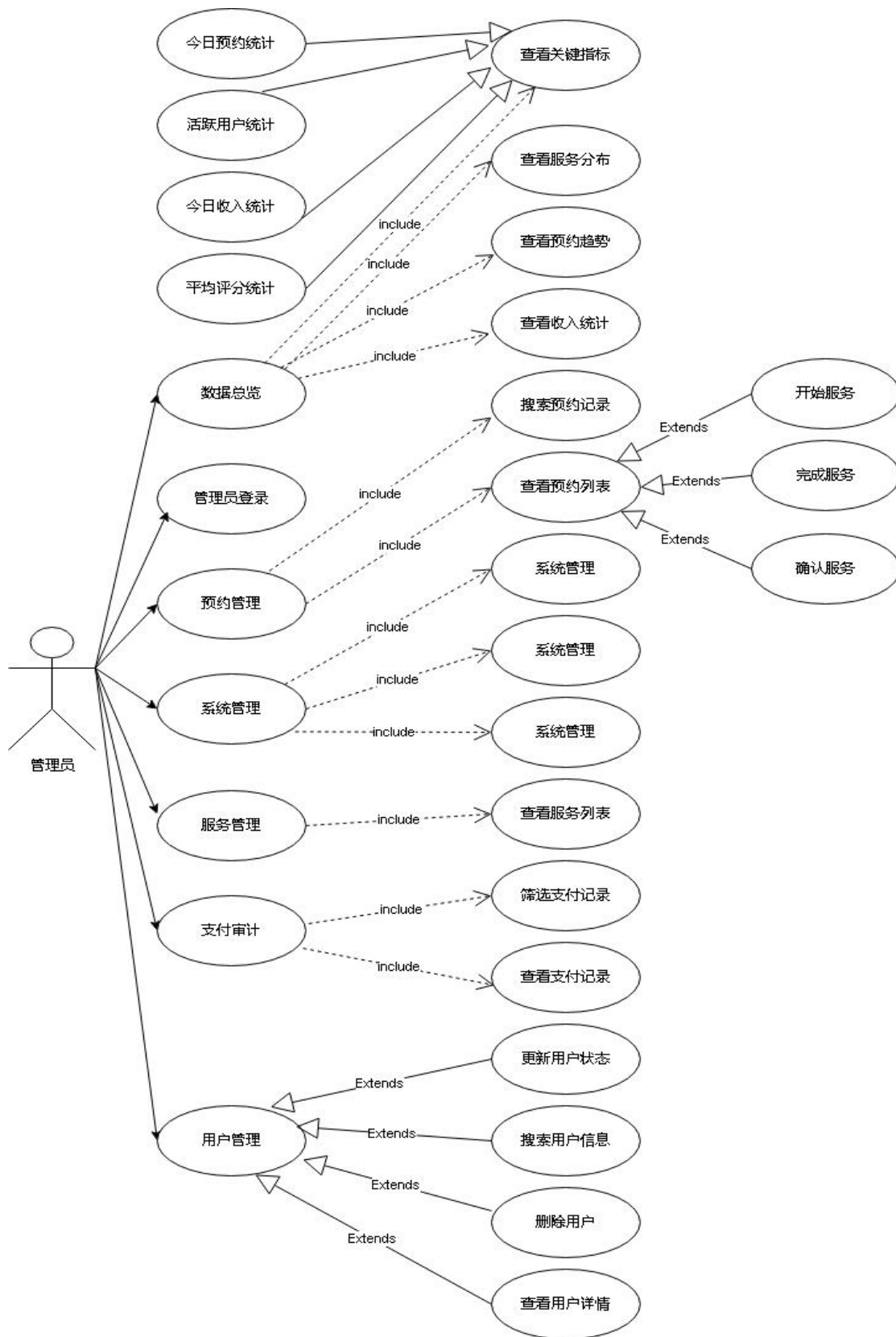


图 3-3 管理员用例图

3.2.2 非功能性需求

1. 性能需求

系统应具备良好的性能表现，具体要求如下：

- (1) 响应时间：系统页面加载时间不超过 3 秒，接口响应时间不超过 2 秒。
- (2) 并发处理：系统应支持至少 100 个用户同时在线访问，峰值时段能够处理更高的并发量。
- (3) 数据库性能：数据库查询响应时间应控制在 2 秒以内，复杂统计查询可适当延长但不超过 5 秒。
- (4) 系统吞吐量：系统应能够处理每秒至少 50 个事务请求。

2. 安全性需求

系统安全性是保障用户信息和业务数据的重要指标，具体要求如下：

- (1) 数据加密：用户密码采用 BCrypt 加密算法存储，支付信息采用 HTTPS 加密传输。
- (2) 身份认证：采用 JWT Token 机制进行用户身份验证，Token 设置有效期，过期后需重新登录。
- (3) 权限控制：系统实现基于角色的访问控制（RBAC），严格区分普通用户和管理员权限。
- (4) 防攻击措施：系统具备 SQL 注入防护、XSS 攻击防护、CSRF 攻击防护等安全机制。
- (5) 支付安全：集成第三方支付平台的安全机制，确保支付交易的安全性。

3. 可靠性需求

系统应具备较高的可靠性，保证业务连续性，具体要求如下：

- (1) 系统可用性：系统可用性应达到 99% 以上，年停机时间不超过 3.65 天。
- (2) 故障恢复：系统应具备自动故障检测和恢复机制，一般故障应在 30 分钟内恢复。
- (3) 数据备份：系统数据应定期备份，数据库采用主从复制机制，确保数据不丢失。
- (4) 异常处理：系统应对各种异常情况进行合理处理，向用户提供友好的错误提示。

4. 易用性需求

系统应具有良好的用户体验，具体要求如下：

- (1) 界面设计：界面简洁美观，符合用户操作习惯，色彩搭配合理。
- (2) 操作便捷：业务流程简洁明了，关键操作不超过 3 步，提供必要的操作引导。
- (3) 错误提示：系统应提供清晰的错误提示信息，帮助用户快速定位和解决问题。
- (4) 多终端适配：系统应支持 PC 端和移动端访问，采用响应式设计自适应不同屏幕尺寸。

3.2.3 数据需求

系统数据流动如图 3-4 和图 3-5 所示，展示了系统与外部实体以及系统内部各模块之间的数据交互关系。

图 3-4 展示了系统与外部实体（用户和管理员）之间的数据流动。用户向系统提交注册信息、预约请求和支付信息，系统处理后返回预约确认、支付结果和订单信息。管理员向系统发送各类管理操作指令，系统返回统计报表和相关数据列表。系统与数据库进行数据的存储和读取交互。

图 3-5 展示了系统内部各业务模块之间的详细数据流动。用户管理模块(1.0)处理用户注册登录，与用户信息库（D1）交互。服务浏览模块（2.0）从服务信息库（D2）读取服务数据。预约管理模块（3.0）接收用户预约请求，查询服务信息库并在订单信息库（D3）中创建订单。支付处理模块（4.0）处理支付请求，更新订单状态并记录支付信息到支付信息库（D4）。数据统计模块（5.0）从各数据库中读取数据，进行汇总分析后提供给管理员。系统主要数据实体包括：

- (1) 用户信息：包括用户 ID、手机号、密码（加密）、姓名、注册时间、账户状态等。
- (2) 服务信息：包括服务 ID、服务名称、描述、价格、时长、服务内容、服务状态等。
- (3) 订单信息：包括订单 ID、订单号、用户 ID、服务 ID、预约日期、预约时间、车牌号、车型、联系方式、订单状态、支付状态等。
- (4) 支付信息：包括支付 ID、支付单号、订单号、用户 ID、支付金额、支付方式、支付状态、支付时间等。
- (5) 时间段信息：包括时间段 ID、日期、开始时间、结束时间、最大预约数、当前预约数等。

第四章 系统设计

4.1 总体设计

4.1.1 功能模块设计

用户认证与管理模块：

这是系统的基础安全模块，负责用户的身份验证和账户管理。该模块支持多种注册方式，包括手机号注册和邮箱注册，并提供完善的密码管理功能。用户可以通过用户名、手机号或验证码等多种方式登录系统，同时支持第三方登录集成。模块还包含个人信息管理功能，用户可以维护自己的基本信息、头像、联系方式等，并支持账户安全设置如密码修改、账号注销等操作。

车辆信息管理模块：

专门用于管理用户的车辆信息，是预约服务的重要基础数据。用户可以添加多辆车辆，包括车牌号、车型、品牌、颜色等详细信息，并可以设置默认车辆以简化预约流程。系统会对车牌号格式进行验证，确保数据的准确性。该模块还提供车辆服务历史记录查询功能，帮助用户了解每辆车的保养情况，并可以设置保养提醒，提升用户体验。

服务项目展示模块：

这是系统的核心业务展示模块，负责向用户展示所有可用的洗车服务项目。模块提供丰富的服务信息展示，包括服务价格、时长、详细介绍和效果图片。支持多维度的服务分类浏览，如基础洗车、精洗套餐、豪华服务等。用户可以通过关键词搜索和多条件筛选（价格区间、服务类型、用户评分等）快速找到合适的服务。同时集成了用户评价系统，展示真实的服务评价和评分，帮助用户做出选择。同时统计‘好评率’‘差评高频词’，结合用户知识隐藏行为研究优化反馈机制^[11]。途虎养车作为行业标杆，其品牌影响力持续提升^[12]，为本系统服务展示模块的商业化设计提供了参考。

智能预约管理模块^{[13][14]}：

这是系统的核心功能模块，实现了智能化的预约流程管理。用户可以选择服务项目、车辆信息，并通过直观的日历界面选择预约日期和时间段。系统具备实时的预约冲突检测功能，确保时间段的合理分配。用户可通过直观的日历界面选择时段，参考网上服务预约模块的核心交互逻辑^[15]。支持预约容量控制，借鉴自

动化车座安排系统的动态资源分配算法^[16]，本模块可优化时段冲突检测效率。同时参考驾校预约系统的流程设计，实现‘日历界面+实时容量提示’的极简交互。避免同一时间段预约过多导致服务质量下降。用户可以在规定时间内修改或取消预约，系统会自动处理相关的业务逻辑，如退款、时间段释放等。

管理后台控制模块：

为管理员提供全面的系统管理功能。包含用户管理、服务管理、预约管理、订单管理等核心业务管理功能。支持细粒度的权限控制，不同角色的管理员具有不同的操作权限。提供系统配置功能，可以设置业务参数、时间段配置、服务容量等。具备完整的日志记录功能，记录所有重要操作，便于问题追踪和审计。还包含数据备份、系统监控等运维功能，确保系统的稳定运行。

4.1.2 系统架构设计

本系统采用 B/S（浏览器/服务器）架构模式，基于经典的三层架构进行设计，如图 4-1 所示。三层架构将系统划分为表示层、业务逻辑层和数据访问层，各层职责明确、边界清晰，层与层之间通过接口进行交互，降低了系统的耦合度，提高了系统的可维护性和可扩展性。

4.2 详细设计

详细设计部分对系统的核心业务模块进行深入设计，描述各模块的业务流程、处理逻辑、关键算法等。本节重点设计用户预约模块、支付模块和订单管理模块三个核心模块。

4.2.1 用户预约模块设计

用户预约模块是系统最核心的业务模块之一，用户通过该模块完成洗车服务的在线预约。用户预约业务流程如图 4-2 所示，详细描述了从用户登录到预约完成的整个流程

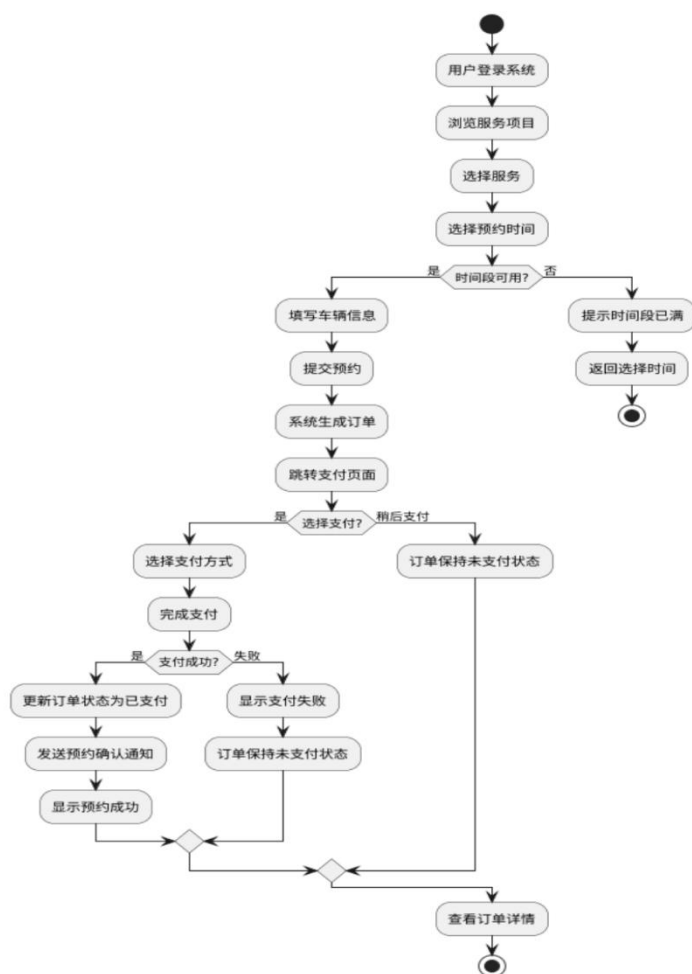


图 4-2 用户预约业务流程图

业务流程说明：

(1) 用户登录系统：用户打开系统首页，如果未登录则跳转到登录页面，输入手机号和密码进行登录。系统验证用户身份后生成 JWT Token 并返回给客户端，客户端将 Token 保存到本地存储（LocalStorage），后续所有请求都携带 Token。

(2) 浏览服务项目：登录成功后，用户进入服务列表页面，系统从服务信息库中查询所有上架状态的服务，按类型分类展示。用户可以查看服务的详细信息，包括价格、时长、服务内容等。

(3) 选择服务：用户点击心仪的服务项目，进入预约页面。系统将选中的服务 ID 传递到预约页面，展示服务的基本信息。

(4) 选择预约时间：用户选择预约日期，系统根据日期查询可用的时间段。查询逻辑为：从时间段表中查询指定日期的所有时间段，计算每个时间段的剩余容量（最大预约数 - 当前预约数），剩余容量大于 0 的时间段标记为可用，剩

余容量为 0 的时间段标记为已满。系统将可用时间段列表返回给客户端，客户端以按钮或卡片形式展示，已满的时间段置灰不可选。

(5) 判断时间段可用性：用户点击某个时间段时，系统再次验证该时间段的可用性。如果时间段已满（可能在用户浏览过程中被其他用户预约），系统提示“该时间段已满，请选择其他时间段”，用户需要重新选择时间段。如果时间段仍然可用，则进入下一步。

(6) 填写车辆信息：用户填写车辆信息表单，包括车牌号（必填）、车型（必填）、车辆颜色（选填）、联系电话（必填）、备注信息（选填）。系统对输入进行前端验证，车牌号需要符合标准格式（如川 B12345），手机号需要为 11 位数字。

(7) 提交预约：用户点击“提交预约”按钮，系统进行后端验证，包括：用户登录状态验证、服务 ID 有效性验证、时间段可用性再次验证（防止并发冲突）、车辆信息完整性和合法性验证。所有验证通过后，系统执行预约创建逻辑。

(8) 系统生成订单：系统在数据库事务中执行以下操作：

生成唯一的订单号（格式为 CW+ 当前时间戳 +4 位随机数，如 CW202501131234567890）。

插入订单记录到订单表，设置订单状态为“待确认”，支付状态为“未支付”，记录用户 ID、服务 ID、时间段 ID、预约日期、预约时间、车辆信息、联系电话、总价格（从服务表获取）、创建时间等。

更新时间段表，将该时间段的当前预约数加 1，防止超额预约。

提交事务，如果任何一步失败则回滚事务。

(9) 转支付页面：订单创建成功后，系统返回订单 ID 和订单号给客户端，客户端跳转到支付页面，传递订单号参数。支付页面展示订单详情和待支付金额。

(10) 选择支付方式：用户在支付页面选择支付方式（微信支付、支付宝

(11) 支付或信用卡支付）。如果用户选择立即支付，则进入支付流程（详见 4.2.2 支付模块设计）。如果用户选择稍后支付，则跳过支付流程。

(12) 支付处理：用户选择支付方式后，系统调用对应的支付接口，生成支付订单，返回支付二维码或跳转 URL。用户完成支付后，第三方支付平台向系统发送异步回调通知。

(13) 判断支付结果：系统接收到支付回调后，验证回调数据的真实性和

完整性，查询支付记录的状态。如果支付成功，执行以下操作：

1. 创建支付记录，保存支付单号、订单号、支付金额、支付方式、支付时间等信息到支付表。
2. 更新订单表，将订单的支付状态设置为“已支付”，记录支付时间和支付方式。
3. 发送预约确认通知给用户，通过短信或站内消息告知预约成功。
4. 跳转到支付成功页面，展示订单信息和支付详情。
5. 如果支付失败，执行以下操作：
6. 不创建支付记录，或创建支付记录并标记状态为“失败”。
7. 订单保持“未支付”状态，用户可以稍后在订单列表中重新支付。
8. 跳转到支付失败页面，展示失败原因，提供重新支付或返回订单列表的按钮。

(14) 查看订单详情：支付完成后，用户可以在订单管理页面查看订单的完整信息，包括订单号、服务名称、预约时间、车辆信息、订单状态、支付状态等。用户可以等待管理员确认订单，或者在需要时取消订单。

关键算法设计：

订单号生成算法：采用“前缀 + 时间戳 + 随机数”的方式生成唯一订单号，前缀为“CW”（CarWash 缩写），时间戳精确到毫秒（13 位），随机数为 4 位数字，总长度为 19 位，确保订单号的唯一性和可读性。

时间段并发控制算法：在更新时间段的当前预约数时，使用数据库的乐观锁或悲观锁机制，防止多个用户同时预约同一时间段导致超额。具体实现为：在 UPDATE 语句中增加 WHERE 条件“current_bookings < max_bookings”，如果更新影响行数为 0，说明时间段已满，抛出异常并回滚事务。类似地，曹思琳（2021）通过 WebSocket 实现售后服务进度推送^[17]，为本系统订单状态同步功能提供了直接参考。Elizaveta K（2021）提出的动态队列分配模型，为本系统高峰期限流策略提供了数学建模参考。

库存扣减策略：采用先查询再更新的两阶段方式，第一阶段查询时间段的可用性，第二阶段在事务中更新时再次验证并扣减库存，避免超卖问题。

异常处理策略：

如果服务已下架，提示“该服务暂时不可用，请选择其他服务”。

如果时间段已满，提示“该时间段已满，请选择其他时间段”。

如果用户未登录，跳转到登录页面并携带重定向参数，登录成功后返回预约页面。

如果网络异常导致预约失败，提示用户检查网络并重试。

如果服务器内部错误，记录详细的错误日志，提示用户“系统繁忙，请稍后再试”。

4.2.2 支付模块设计

支付模块负责处理用户的在线支付请求，集成第三方支付平台，确保支付的安全性和可靠性。支付处理流程如图 4-3 所示，展示了支付的完整业务流程。

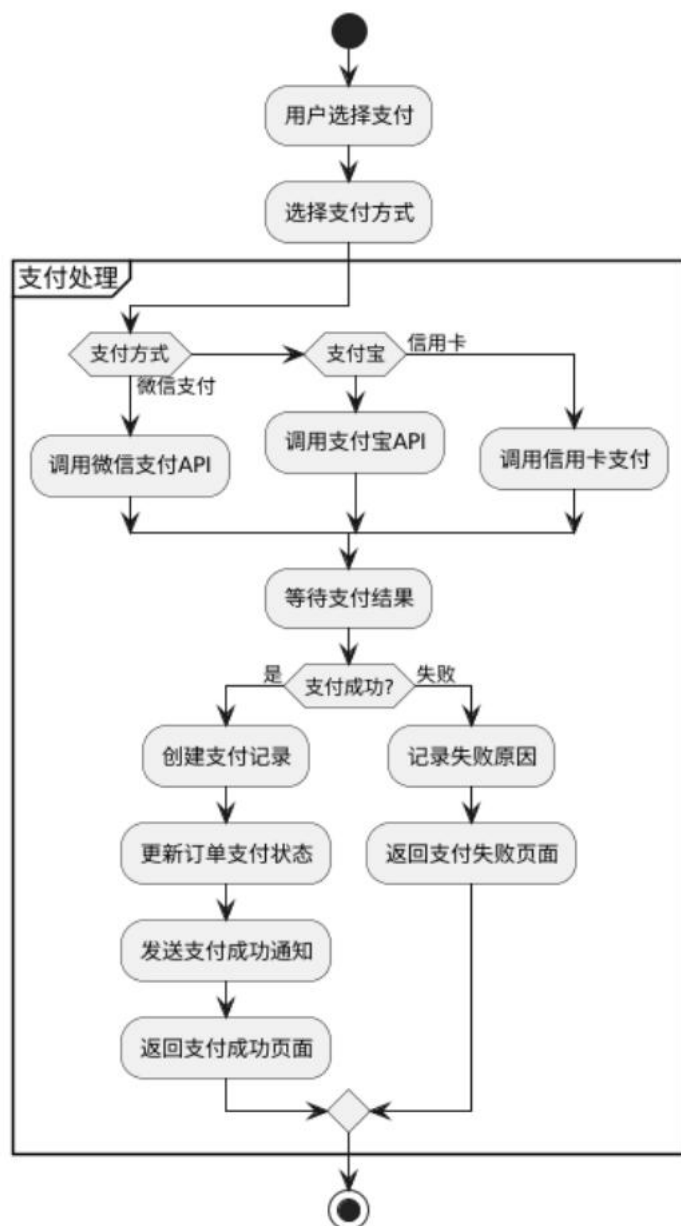


图 4-3 支付处理流程图

业务流程说明：

(1) 用户选择支付：用户在订单详情页或支付页面点击“立即支付”按钮，进入支付流程。系统首先验证订单的支付状态，如果订单已支付，提示“订单已支付，无需重复支付”并返回。如果订单未支付，则继续支付流程。

(2) 选择支付方式：系统展示支持的支付方式列表，包括微信支付、支付宝支付、信用卡支付。每种支付方式展示对应的图标和说明。用户点击选择其中一种支付方式。

(3) 调用支付接口：根据用户选择的支付方式，系统调用不同的支付接口：

微信支付：调用微信支付统一下单 API (pay/unifiedorder)，传递商户号、订单号、订单金额、回调 URL 等参数。微信支付返回预支付交易 ID (prepay_id) 和支付二维码字符串。系统生成二维码图片展示给用户，用户使用微信扫一扫完成支付。

支付宝支付：调用支付宝手机网站支付接口 (alipay.trade.wap.pay) 或电脑网站支付接口 (alipay.trade.page.pay)，传递商户 ID、订单号、订单金额、回调 URL 等参数。支付宝返回支付表单 HTML 或支付 URL。系统将用户跳转到支付宝支付页面，用户使用支付宝账户或扫码完成支付。

信用卡支付：调用银行或第三方支付平台的信用卡支付接口，传递订单信息和信用卡信息（卡号、有效期、CVV 码）。接口返回支付结果或跳转 URL。用户输入信用卡信息并提交，银行进行验证后完成支付。

(4) 创建支付订单：系统在调用第三方支付接口前，先在支付表中创建支付记录，记录支付单号（系统生成的唯一标识）、订单号、用户 ID、支付金额、支付方式、支付状态（设置为“待支付”）、创建时间等信息。支付单号的生成规则为：“PAY + 时间戳 + 随机数”，如 PAY2025011312345678901234。

(5) 等待支付结果：用户在第三方支付平台完成支付操作后，第三方平台会向系统发送异步回调通知 (Webhook)，通知支付结果。系统提供回调接口接收通知，如 /api/payment/callback/wechat、/api/payment/callback/alipay 等。

(6) 判断支付是否成功：系统接收到回调通知后，进行以下验证：
验证回调数据的签名，确保数据来自合法的支付平台，防止伪造。

验证订单号和支付金额是否与系统记录一致，防止数据篡改。

查询支付记录的当前状态，如果已处理则忽略重复通知（支付平台可能多次发送回调）。

验证支付状态，如果为“成功”则执行支付成功逻辑，如果为“失败”则执行支付失败逻辑。

(7) 支付成功处理：如果支付成功，系统在数据库事务中执行以下操作：

更新支付记录：将支付状态设置为“已支付”，记录第三方交易号、支付时间等信息。

更新订单记录：将订单的支付状态设置为“已支付”，记录支付时间和支付方式。

发送支付成功通知：调用短信服务接口或消息推送接口，向用户发送支付成功通知，包含订单号和预约信息。

返回支付结果：向客户端返回支付成功的响应，客户端跳转到支付成功页面，展示“支付成功！您的预约已确认”等提示信息，显示订单详情和二维码（用于到店核销）。

(8) 支付失败处理：如果支付失败，系统执行以下操作：

1. 更新支付记录：将支付状态设置为“支付失败”，记录失败原因（如余额不足、银行卡过期、密码错误等）。

2. 订单状态保持不变：订单的支付状态仍为“未支付”，用户可以重新发起支付。

3. 返回支付结果：向客户端返回支付失败的响应，客户端跳转到支付失败页面，展示失败原因和提示信息，提供“重新支付”和“返回订单列表”两个按钮。

用户点击“重新支付”可以重新选择支付方式并发起支付。点击“返回订单列表”可以返回到订单管理页面，稍后再进行支付。关键技术实现：

4. 支付安全：所有支付相关的数据传输采用 HTTPS 协议加密。支付接口调用时对参数进行签名，第三方平台验证签名后才处理请求。基于 RESTful API 的访问权限系统设计^[18]确保了接口调用的安全性。接收回调通知时验证签名，确保数据来源可信。用户的信用卡信息不保存在系统中，符合 PCI DSS 安全标准。

5. 幂等性保证：支付回调接口实现幂等性，即使第三方平台多次发送回调通知，系统也只处理一次。实现方式为：在处理回调前先查询支付记录的状态，如果已经是“已支付”状态则直接返回成功，不重复处理。

6. 异步通知处理：支付回调是异步的，用户完成支付后系统可能还未收到回调。因此前端需要实现轮询机制，每隔几秒查询一次订单的支付状态，直到支付成功或超时。超时时间设置为 5 分钟，超时后提示用户“支付结果确认中，请稍后在订单列表中查看”。

7. 支付单号与订单号的关系：一个订单只对应一个成功的支付记录，但可能存在多次支付尝试。系统记录所有的支付记录，通过订单号关联订单，通过支付状态区分成功和失败的支付。

8. 异常处理策略：如果调用支付接口失败（网络异常、接口返回错误等），记录详细的错误日志，提示用户“支付服务暂时不可用，请稍后重试”。

如果支付金额与订单金额不符，拒绝支付并记录异常日志，提示用户“订单金额异常，请联系客服”。

如果回调数据验签失败，记录可疑请求的 IP 和数据，拒绝处理并返回失败。

如果支付超时未收到回调，订单保持“未支付”状态，用户可以在订单列表中查看并重新支付。

4.2.3 订单管理模块设计

订单管理模块是管理员处理预约订单的核心模块，负责订单的审核、状态更新、取消、删除等操作。订单处理流程如图 4-4 所示，展示了管理员处理订单的完整业务流程。

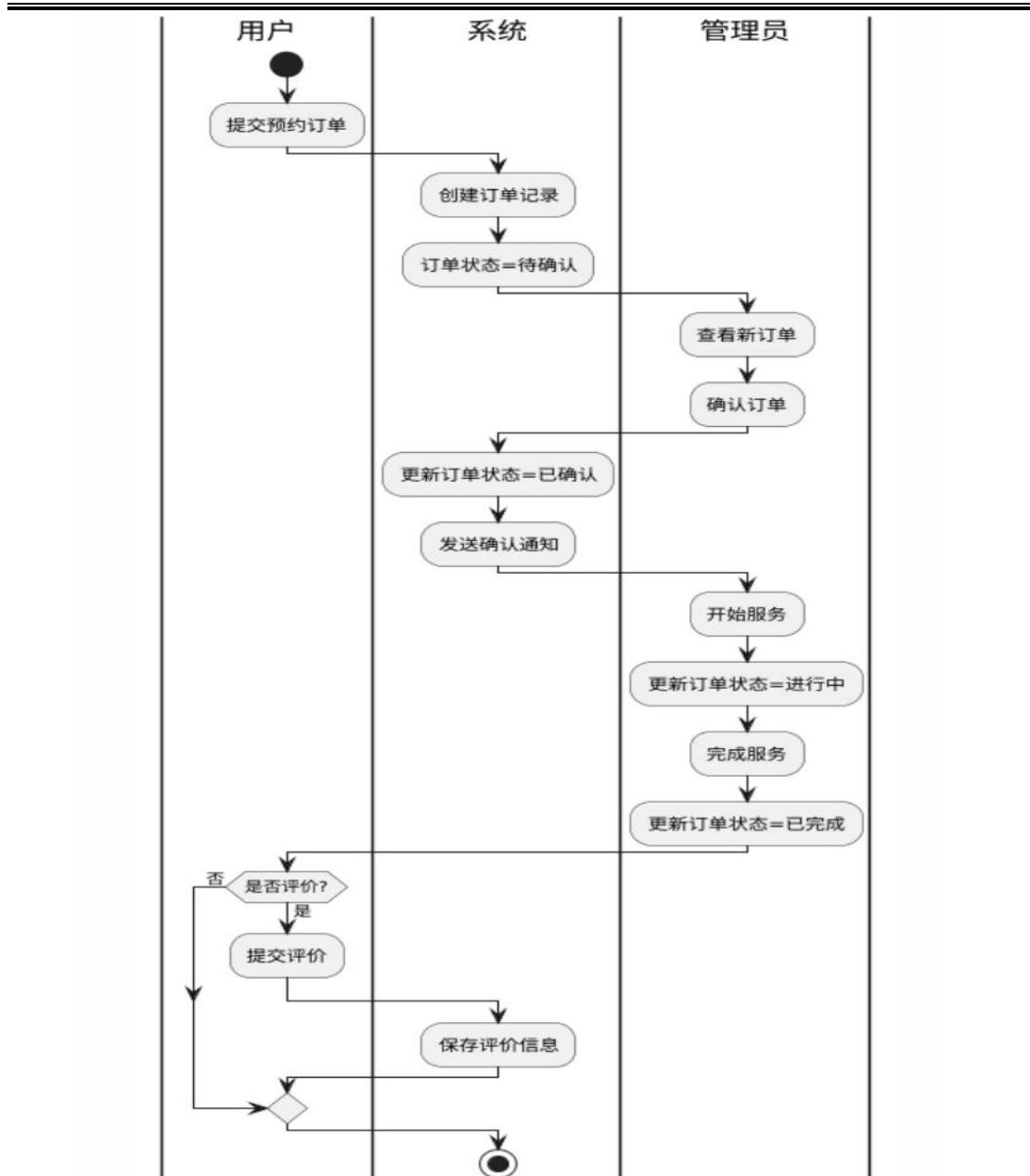


图 4-4 订单处理流程图

1. 业务流程说明:

(1) 用户提交预约订单: 用户在前端完成预约流程后, 系统创建订单记录并保存到订单表, 订单状态初始化为“待确认”。同时, 系统可以通过 WebSocket 或消息队列向管理端推送新订单通知, 提醒管理员及时处理。

(2) 管理员查看新订单: 管理员登录后台管理系统, 进入预约管理模块。系统展示所有订单的列表, 管理员可以通过状态筛选查看“待确认”的订单。订单列表按创建时间倒序排列, 最新的订单显示在最前面, 方便管理员优先处理。

(3) 管理员确认订单: 管理员点击订单详情, 查看用户的预约信息,

包括服务项目、预约时间、车辆信息、联系电话等。审核无误后，管理员点击“确认订单”按钮，系统执行确认操作。

(4) 更新订单状态为“已确认”：系统在数据库中将订单状态更新为“已确认”，同时记录确认时间和确认人（管理员 ID）。订单状态的变更会记录到操作日志表中，形成审计轨迹。

(5) 发送确认通知：系统调用短信服务接口，向用户发送预约确认通知。短信内容包括：“尊敬的用户，您预约的服务名称已确认，预约时间为预约时间，地址为服务地址，请准时到达。如有疑问请致电客服电话。”用户收到短信后可以做好准备，按时到店享受服务。

(6) 管理员开始服务：到达预约时间后，用户到店，服务人员开始提供服务。管理员在后台系统中点击“开始服务”按钮，系统将订单状态更新为“进行中”，记录服务开始时间。此时订单不可取消，用户在前端可以看到订单状态的变化。

(7) 管理员完成服务：服务结束后，管理员点击“完成服务”按钮，系统将订单状态更新为“已完成”，记录服务完成时间。

(8) 系统发送完成通知：系统向用户发送服务完成通知，提醒用户可以对服务进行评价。通知内容为：“您的服务名称已完成，感谢您的使用！欢迎您对本次服务进行评价，帮助我们改进服务质量。”

(9) 用户提交评价：用户收到通知后，可以在订单详情页面进行评价。评价内容包括服务质量评分、服务态度评分、环境评分、文字评价等。用户提交评价后，系统将评价数据保存到评价表中，关联订单 ID 和用户 ID。

(10) 系统保存评价信息：系统将评价数据插入到评价表，同时可以计算服务的平均评分，更新到服务表中，为其他用户提供参考。评价提交后，订单流程完整结束。

2. 订单取消流程：如果订单需要取消（用户主动取消或管理员取消），流程如下：

(1) 发起取消请求：用户或管理员点击“取消订单”按钮，系统弹出取消原因选择框，包括“时间冲突”、“计划变更”、“服务不满意”、“其他原因”等选项。用户或管理员选择原因并填写详细说明（可选）。

(2) 判断订单状态：系统验证订单的当前状态，只有“待确认”和“

已确认”状态的订单允许取消。如果订单状态为“进行中”或“已完成”，系统拒绝取消并提示“订单已开始服务或已完成，无法取消”。

(3) 更新订单状态为“已取消”：系统将订单状态更新为“已取消”，记录取消时间、取消人、取消原因。同时，将该订单占用的时间段释放，即将时间段表中对应时间段的当前预约数减 1，使该时间段重新可用。

(4) 处理退款：如果订单已支付，系统需要发起退款流程。调用第三方支付平台的退款接口，传递原支付单号、退款金额、退款原因等参数。支付平台处理退款后，向系统发送退款结果回调。系统更新支付记录的状态为“已退款”，记录退款时间。退款金额原路返回用户的支付账户，通常在 1-7 个工作日内到账。

(5) 发送取消通知：系统向用户发送订单取消通知，告知订单已取消，如已支付则说明退款将在几个工作日内到账。

3. 关键算法设计：

订单状态流转控制：订单状态的流转遵循固定的流程：“待确认” → “已确认” → “进行中” → “已完成”，或在“待确认”、“已确认”阶段跳转到“已取消”。系统在状态更新时验证当前状态是否允许转换到目标状态，防止非法状态跳转。

时间段库存回滚：订单取消时需要将时间段的预约数减 1，实现库存回滚。为防止并发问题，使用数据库事务和行锁，确保库存的准确性。

操作日志记录：所有对订单的操作（创建、确认、开始、完成、取消、删除）都记录到操作日志表，包含操作人、操作时间、操作类型、操作前状态、操作后状态、备注等字段，便于审计和问题追溯。

4. 异常处理策略：

如果确认订单时订单已被其他管理员确认，提示“订单已确认，无需重复操作”。

如果取消订单时订单已被取消，提示“订单已取消，无需重复操作”。

如果退款接口调用失败，记录错误日志，提示管理员“自动退款失败，请手动处理退款”，并标记订单为“待退款”状态。业务逻辑漏洞常见类型分析研究^[19]为系统支付审计和异常处理提供了全面的安全验证方案。

如果网络异常导致操作失败，提示用户检查网络并重试。

如果订单不存在或已删除，提示“订单不存在或已被删除”。

4.3 数据库设计

4.3.1 数据库概念设计

数据库概念设计采用实体-关系模型，基于响应式设计的移动端 UI 动态适配框架研究^[20]为多终端数据展示提供了界面设计参考。

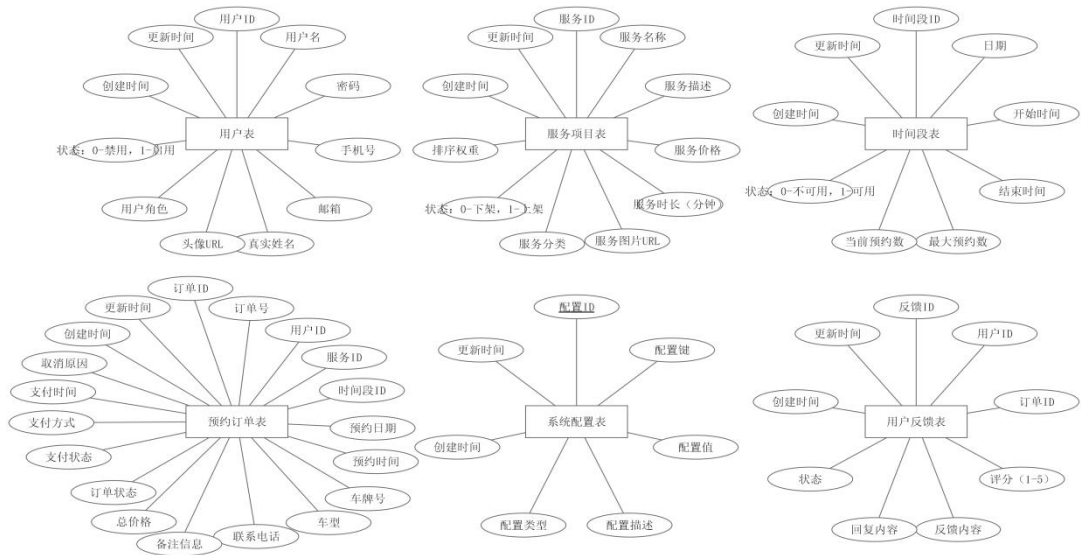


图 4-5 数据库 E-R 图

4.3.2 数据库逻辑设计

表 4-1 用户表 (users) 结构

字段名	数据类型	长度	是否为空	默认值	说明
id	BIGINT	-	NOT NULL	自增	用户 ID，主键
phone	VARCHAR	20	NOT NULL	-	手机号，唯一索引
password	VARCHAR	255	NOT NULL	-	密码（加密存储）
name	VARCHAR	50	NULL	NULL	姓名
gender	VARCHAR	10	NULL	NULL	性别
role	VARCHAR	20	NOT NULL	'user'	角色（user/admin）
tatus	INT	-	NOT NULL	1	账户状态（1-正常，0-禁用）
created_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	注册时间

last_login_at	TIMESTAMP	-	NULL	NULL	最后登录时间
---------------	-----------	---	------	------	--------

表 4-2 服务表（services）结构

字段名	数据类型	长度	是否为空	默认值	说明
id	BIGINT	-	NOT NULL	自增	服务 ID，主键
name	VARCHAR	100	NOT NULL	-	服务名称
type	VARCHAR	50	NOT NULL	-	服务类型
description	TEXT	-	NULL	NULL	服务描述
price	DECIMAL	10,2	NOT NULL	-	价格（单位：元）
duration	INT	-	NOT NULL	-	时长（单位：分钟）
content	TEXT	-	NULL	NULL	服务内容详情
status	VARCHAR	20	NOT NULL	'active'	服务状态 (active-上架，inactive-下架)
created_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	创建时间
updated_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	TINYINT	-	NOT NULL	0	逻辑删除标记

表 4-3 订单表 (bookings) 结构

字段名	数据类型	长度	是否为空	默认值	说明
id	BIGINT	-	NOT NULL	自增	订单 ID, 主键
order_no	VARCHAR	50	NOT NULL	-	订单号, 唯一索引
user_id	BIGINT	-	NOT NULL	-	用户 ID, 外键
service_id	BIGINT	-	NOT NULL	-	服务 ID, 外键
time_slot_id	BIGINT	-	NOT NULL	-	时间段 ID, 外键
booking_date	DATE	-	NOT NULL	-	预约日期
booking_time	VARCHAR	20	NOT NULL	-	预约时间
car_number	VARCHAR	20	NULL	NULL	车牌号
car_model	VARCHAR	50	NULL	NULL	车型
car_color	VARCHAR	20	NULL	NULL	车辆颜色
contact_phone	VARCHAR	20	NOT NULL	-	联系电话
notes	TEXT	-	NULL	NULL	备注信息
total_price	DECIMAL	10,2	NOT NULL	-	总价格
status	VARCHAR	20	NOT NULL	'pending'	订单状态
payment_status	VARCHAR	20	NOTNULL	'unpaid'	支付状态
payment_method	VARCHAR	20	NULL	NULL	支付方式
paid_at	TIMESTAMP	-	NULL	NULL	支付时间
completed_at	TIMESTAMP	-	NULL	NULL	完成时间

cancelled_at	TIMESTAMP	-	NULL	NULL	取消时间
cancel_reason	VARCHAR	255	NULL	NULL	取消原因

续 4-3

字段名	数据类型	长度	是否为空	默认值	说明
created_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	创建时间
updated_at	TIMESTAMP	-	NOTNULL	CURRENT_TIMESTAMP	更新时间
deleted	TINYINT	-	NOTNULL	0	逻辑删除 标记

表 4-4 支付表（payments）结构

字段名	数据类型	长度	是否为空	默认值	说明
id	BIGINT	-	NOT NULL	自增	支付 ID，主键
payment_no	VARCHAR	50	NOT NULL	-	支付单号，唯一索引
order_no	VARCHAR	50	NOT NULL	-	订单号，关联订单
user_id	BIGINT	-	NOT NULL	-	用户 ID，外键
amount	DECIMAL	10,2	NOT NULL	-	支付金额
payment_method	VARCHAR	20	NOT NULL	-	支付方式（wechat/alipay/credit_card）
status	VARCHAR	20	NOT NULL	'pending'	支付状态
transaction_id	VARCHAR	100	NULL	NULL	第三方交易号
paid_at	TIMESTAMP	-	NULL	NULL	支付时间
refund_at	TIMESTAMP	-	NULL	NULL	退款时间
refund_reason	VARCHAR	255	NULL	NULL	退款原因
created_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	创建时间

updated_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	更新时间
------------	-----------	---	----------	-------------------	------

表 4-5 时间段表 (time_slots) 结构

字段名	数据类型	长度	是否为空	默认值	说明
id	BIGINT	-	NOT NULL	自增	时间段 ID, 主键
date	DATE	-	NOT NULL	-	日期
start_time	TIME	-	NOT NULL	-	开始时间
end_time	TIME	-	NOT NULL	-	结束时间
max_bookings	INT	-	NOT NULL	5	最大预约数
current_bookings	INT	-	NOT NULL	0	当前预约数
created_at	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	创建时间

第五章 系统实现

5.1 开发环境搭建

本系统的开发环境包括前端开发环境和后端开发环境，具体配置如下：

(1) 硬件环境

开发主机：CPU Intel Core i5 或以上，内存 8GB 或以上，硬盘可用空间 50GB 以上

服务器：云服务器或本地服务器，CPU 2 核以上，内存 4GB 以上

(2) 软件环境前端开发环境：

操作系统：Windows 10/11 或 macOS 或 Linux

Node.js：版本 18.x 或更高

npm：版本 9.x 或更高（随 Node.js 安装）

代码编辑器：Visual Studio Code 或 WebStorm

浏览器：Chrome 或 Edge（用于调试）

后端开发环境：

操作系统：Windows 10/11 或 macOS 或 Linux

JDK：Java Development Kit 17 或更高

Maven：版本 3.8.x 或更高（用于项目构建和依赖管理）

IDE：IntelliJ IDEA 或 Eclipse

数据库：MySQL 8.0 或更高

缓存：Redis 5.x 或更高

(3) 开发工具

版本控制：Git

API 测试工具：Postman 或 Apifox

数据库管理工具：Navicat 或 MySQL Workbench

Redis 管理工具：RedisInsight 或命令行工具

5.2 关键技术实现

5.2.1 用户认证与授权实现

系统采用 JWT（JSON Web Token）机制实现用户认证与授权。具体实现如下：

(1) JWT Token 生成

用户登录成功后，系统生成 JWT Token 并返回给客户端。Token 包含用户 ID、角色等信息，使用密钥进行签名。

```
// 生成JWT Token
public String generateToken(User user) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("userId", user.getId());
    claims.put("role", user.getRole());

    return Jwts.builder()
        .setClaims(claims)
        .setSubject(user.getPhone())
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
        .signWith(SignatureAlgorithm.HS512, SECRET_KEY)
        .compact();
}
```

(2) JWT Token 验证

客户端每次请求携带 Token，后端通过拦截器验证 Token 的有效性。

```
// Token验证拦截器
public boolean preHandle(HttpServletRequest request,
                        HttpServletResponse response,
                        Object handler) {
    String token = request.getHeader("Authorization");
    if (token != null && token.startsWith("Bearer ")) {
        token = token.substring(7);
        try {
            Claims claims = Jwts.parser()
                .setSigningKey(SECRET_KEY)
                .parseClaimsJws(token)
                .getBody();

            Long userId = claims.get("userId", Long.class);
            String role = claims.get("role", String.class);

            // 将用户信息存入请求上下文
            request.setAttribute("userId", userId);
            request.setAttribute("role", role);
            return true;
        } catch (Exception e) {
            response.setStatus(401);
            return false;
        }
    }
    response.setStatus(401);
    return false;
}
```

(3) 权限控制

系统通过自定义注解和 AOP 实现细粒度的权限控制。

```

// 权限注解
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface RequiresRole {
    String value(); // "user" 或 "admin"
}

// 权限切面
@Aspect
@Component
public class AuthorizationAspect {
    @Before("@annotation(requiresRole)")
    public void checkPermission(JoinPoint joinPoint, RequiresRole requiresRole) {
        HttpServletRequest request = getCurrentRequest();
        String userRole = (String) request.getAttribute("role");

        if (!requiresRole.value().equals(userRole)) {
            throw new UnauthorizedException("权限不足");
        }
    }
}

```

5.2.2 支付接口集成实现

系统集成支付宝支付接口，实现在线支付功能。

(1) 支付宝 SDK 配置

(2) 创建支付订单

(3) 支付回调处理系统接收到支付回调后，需验证签名和金额一致性，

RESTful API 权限控制系统为支付接口^[17]的权限管理提供了技术实现方案。

5.2.3 数据缓存实现

```

@Service
public class ServiceCacheService {
    @Autowired
    private RedisTemplate<String, Object> redisTemplate;

    @Autowired
    private ServiceMapper serviceMapper;

    private static final String CACHE_KEY_PREFIX = "service:";
    private static final long CACHE_EXPIRATION = 3600; // 1小时

    public List<Service> getServiceList() {
        String cacheKey = CACHE_KEY_PREFIX + "list";

        // 先从缓存获取
        List<Service> services = (List<Service>) redisTemplate.opsForValue().get(cacheKey);

        if (services == null) {
            // 缓存未命中，从数据库查询
            services = serviceMapper.selectAll();

            // 写入缓存
            redisTemplate.opsForValue().set(cacheKey, services, CACHE_EXPIRATION, TimeUnit.SECONDS);
        }

        return services;
    }

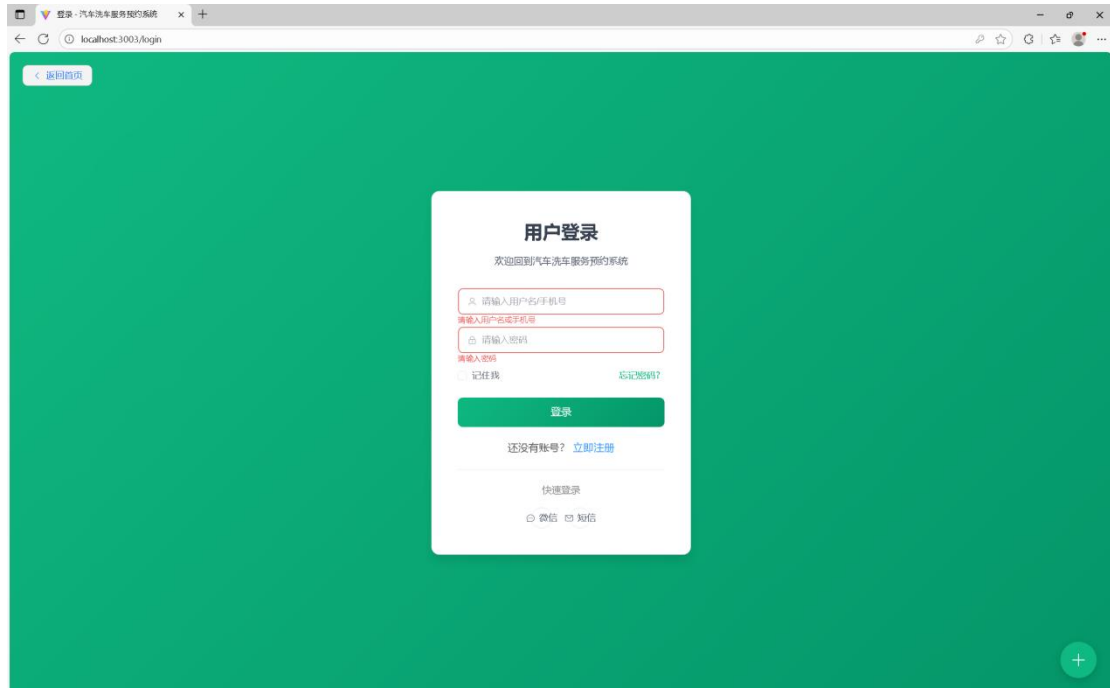
    public void clearCache() {
        Set<String> keys = redisTemplate.keys(CACHE_KEY_PREFIX + "*");
        if (keys != null && !keys.isEmpty()) {
            redisTemplate.delete(keys);
        }
    }
}

```

5.3 主要功能实现

5.3.1 用户端功能实现

用户注册登录界面及功能实现



注册 - 汽车洗车服务预约系统 x +

localhost:3003/register

< 返回首页

用户注册

加入汽车洗车服务预约系统

请输入用户名

请输入真实姓名

请输入手机号

请输入验证码 [获取验证码](#)

请输入邮箱

请输入密码

请确认密码

[注册](#)

已有账号? [立即登录](#)

+

注册 - 汽车洗车服务预约系统 x +

localhost:3003/register

< 返回首页

用户注册

加入汽车洗车服务预约系统

Dufu

姓由

13030303030

请输入验证码 [获取验证码](#)

Dufu@123.com

A@123456

A@123456

[注册](#)

已有账号? [立即登录](#)

+

注册 - 汽车洗车服务预约系统 x +

localhost:3003/login

< 返回首页

注册成功! 请登录

用户登录

欢迎来到汽车洗车服务预约系统

user

☐ 记住我 [忘记密码?](#)

[登录](#)

还没有账号? [立即注册](#)

快速登录

微信 短信

保存密码?

你的密码将在下次自动填充, 并保存在你的设备

用户名 Dufu@123.com

密码 *****

[保存](#) [以后再说](#)

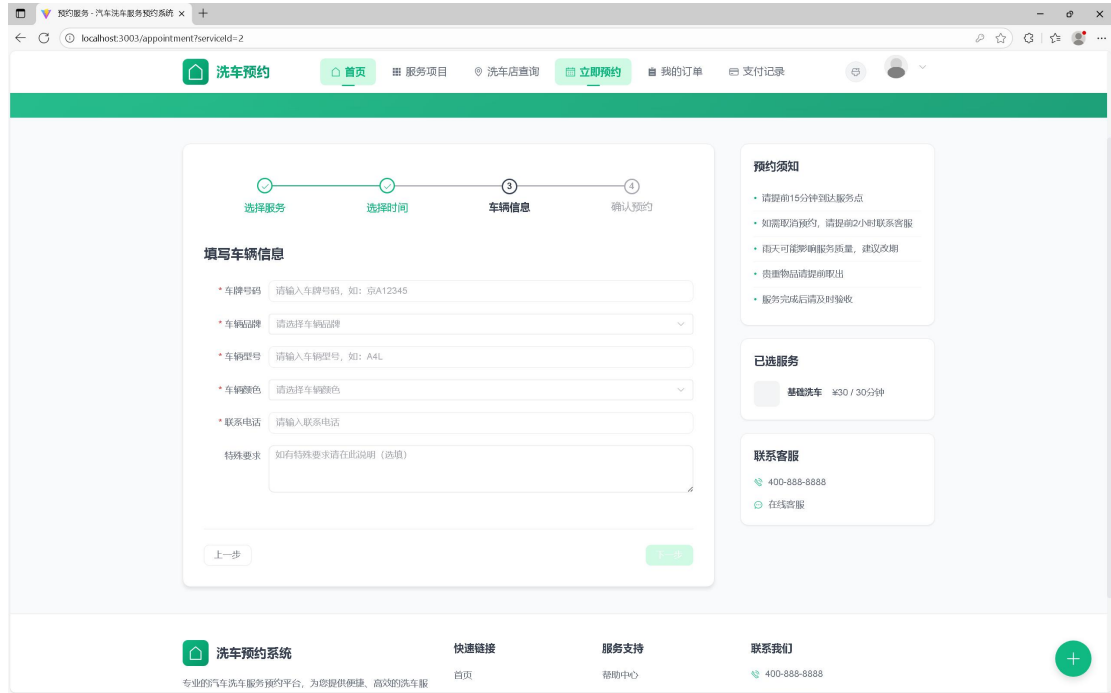
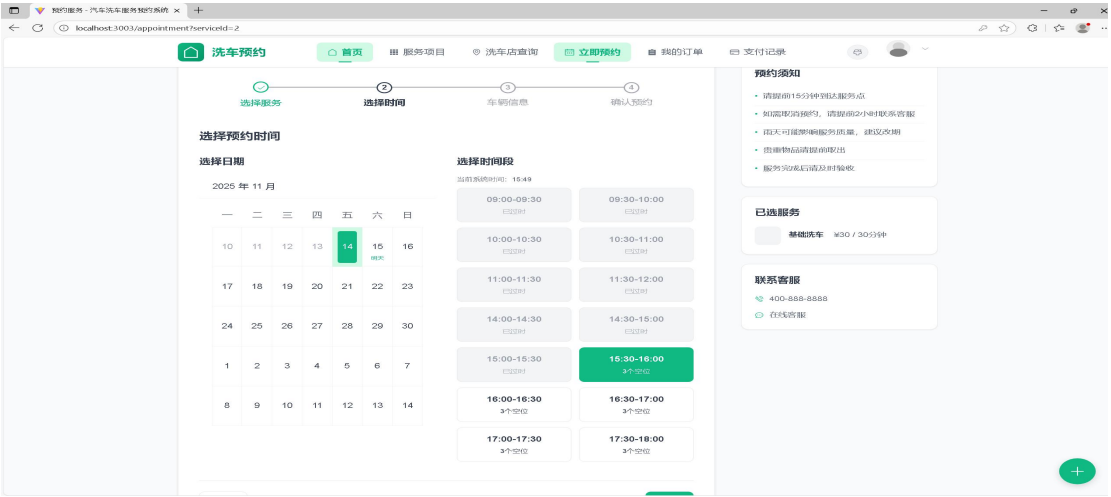
+

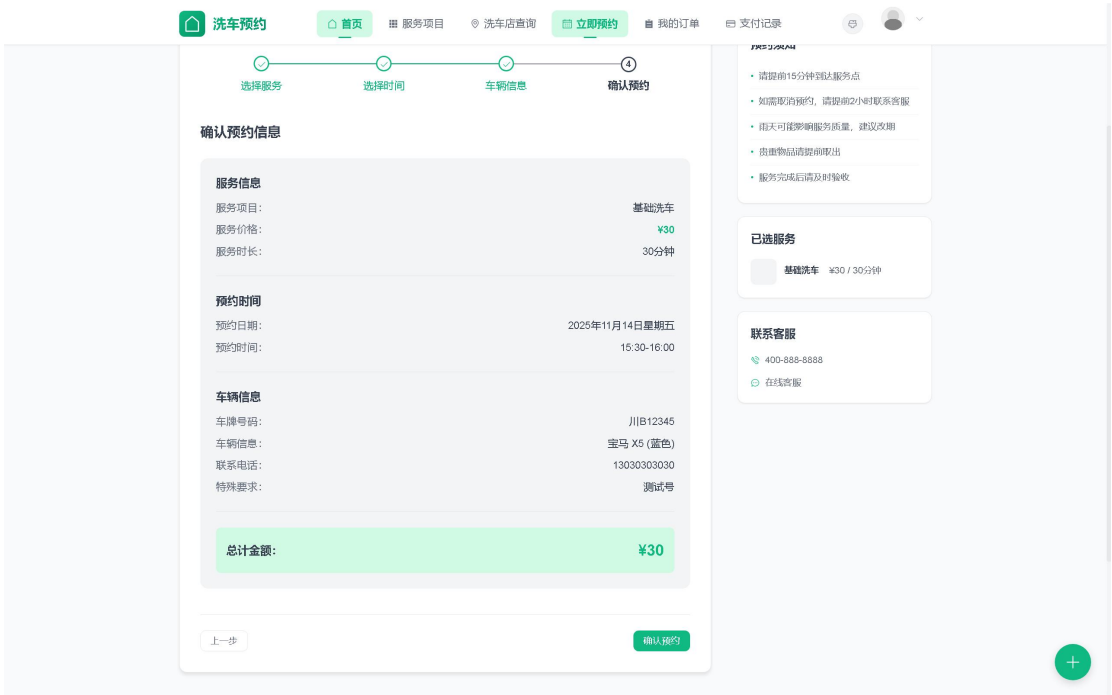


服务浏览界面及功能实现

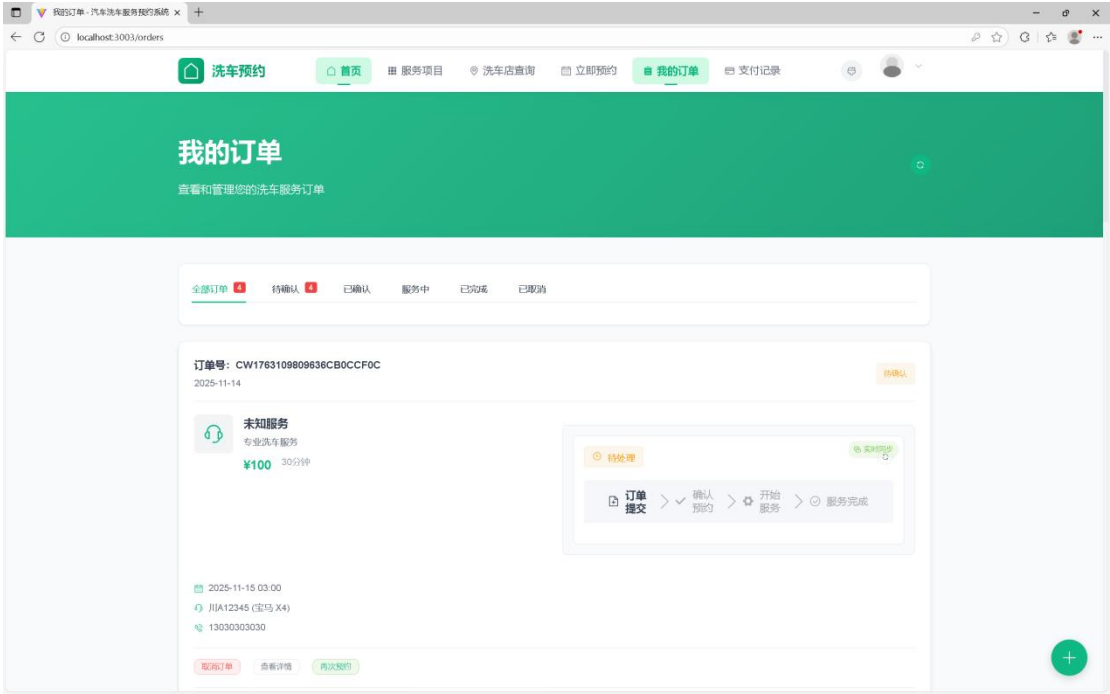


在线预约界面及功能实现

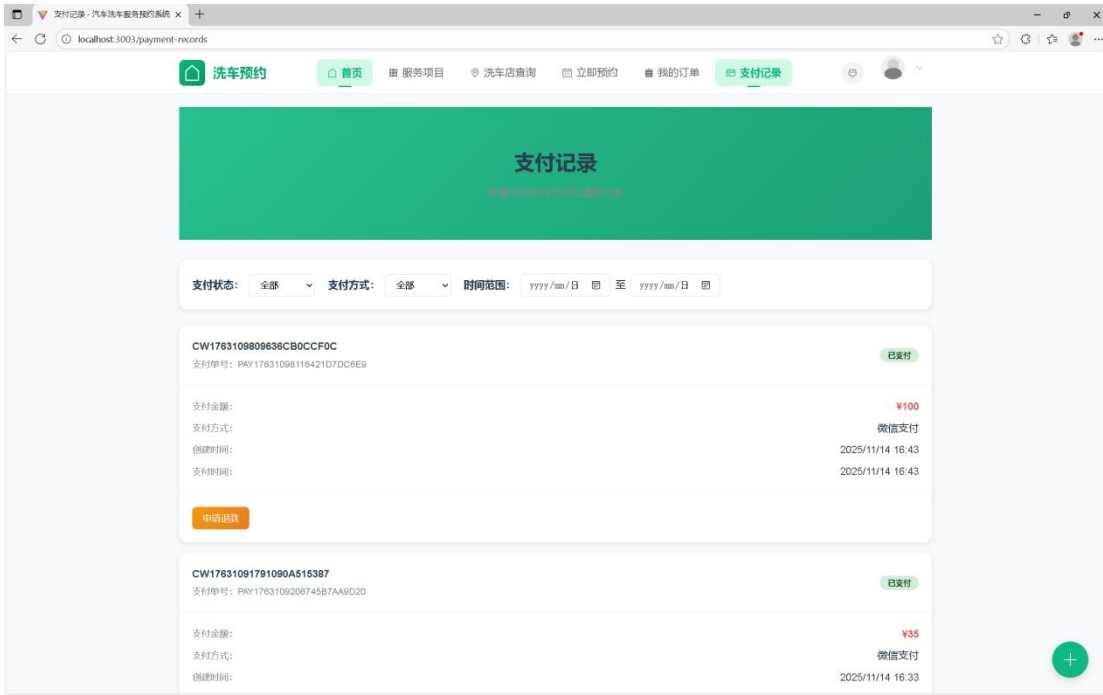
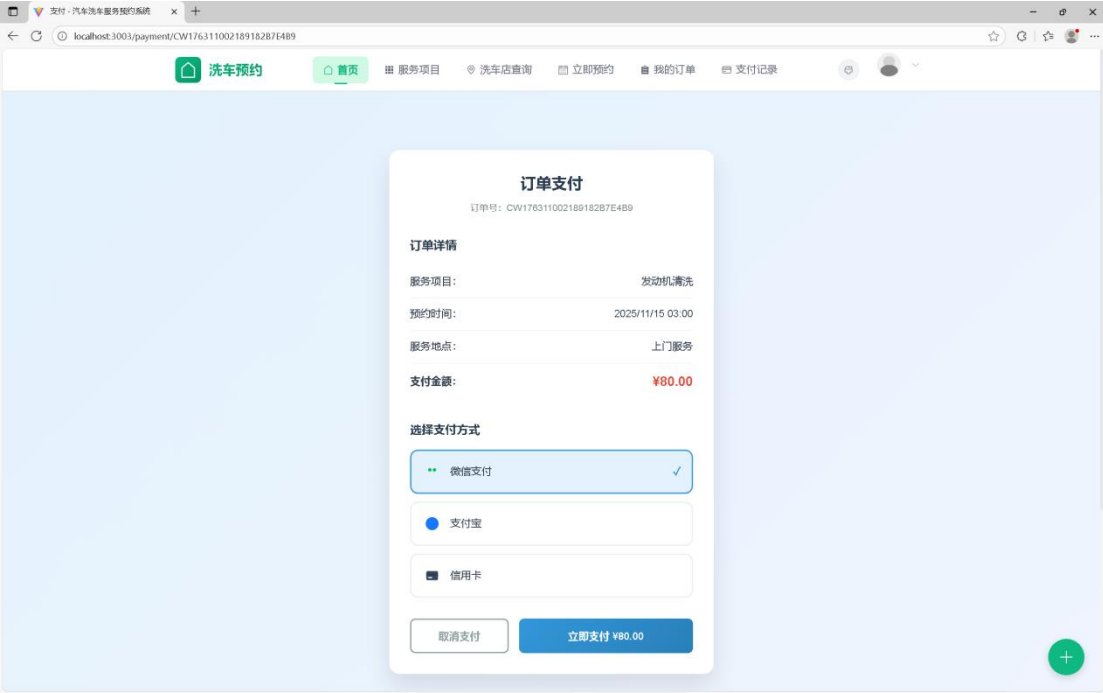




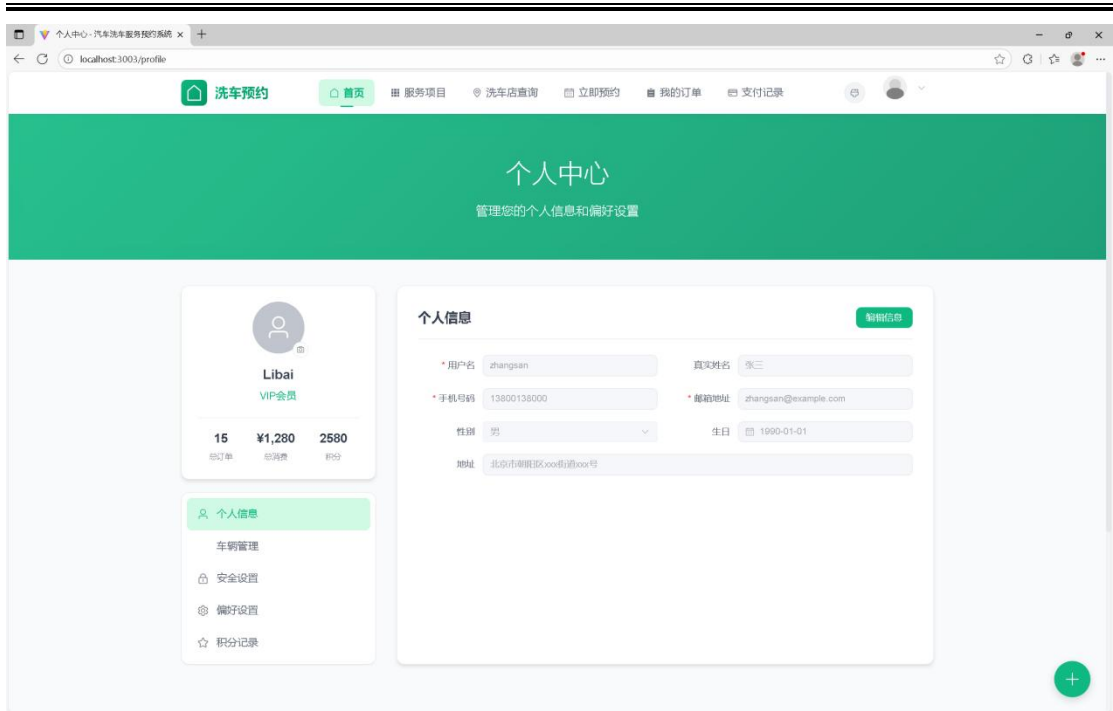
订单管理界面及功能实现



支付界面及功能实现



个人中心界面及功能实现

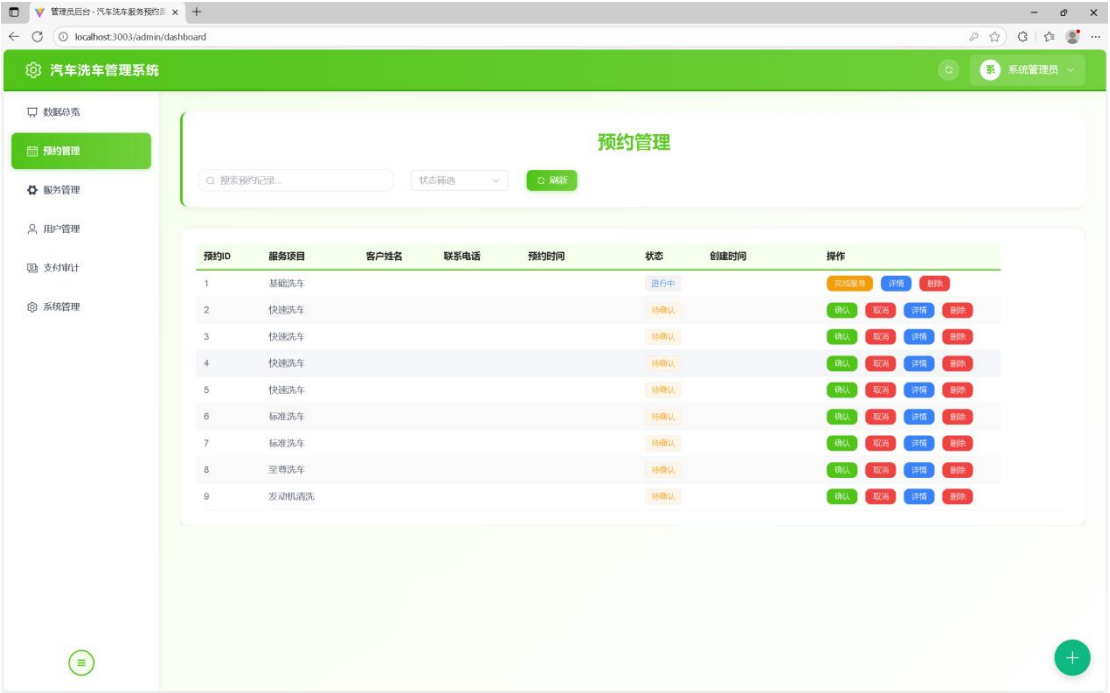


5. 3. 2 管理端功能实现

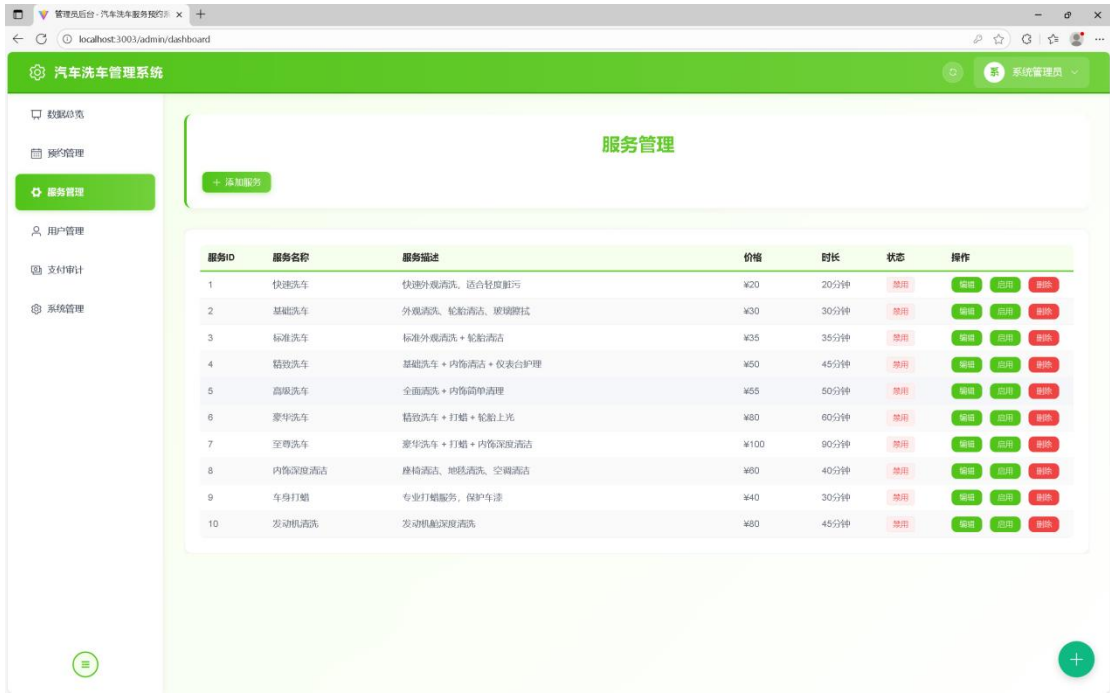
数据统计界面及功能实现



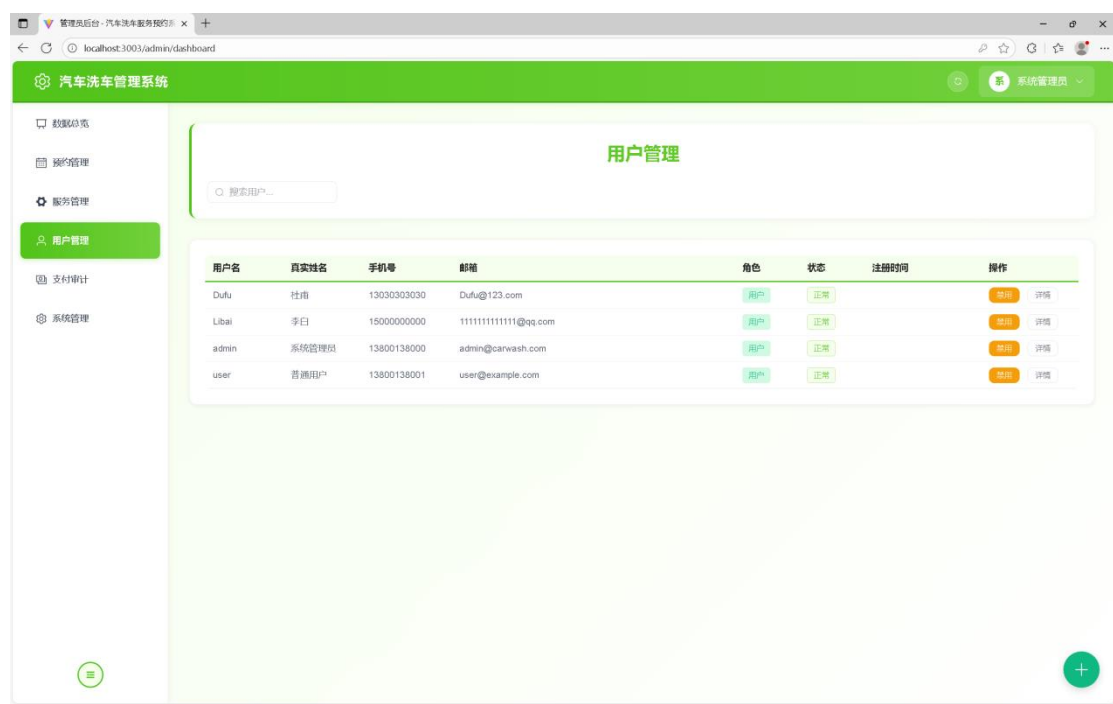
预约管理界面及功能实现



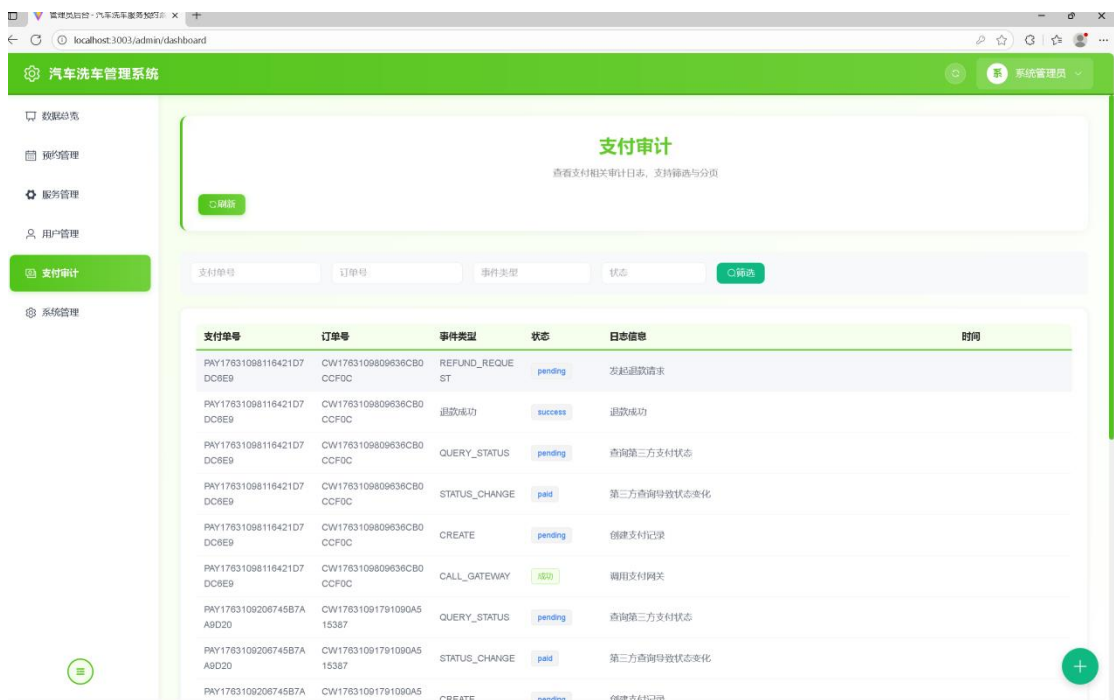
服务管理界面及功能实现



用户管理界面及功能实现



支付审计界面及功能实现



第六章 系统测试

6.1 功能测试

6.1.1 用户注册功能测试

用户注册是用户使用系统的第一步，需要验证注册流程的正确性和数据验证的有效性。测试用例如表 6-1 所示。

表 6-1 用户注册功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果
TC-REG-01	正常注册	手机号: 13800138000		
		密码: 123456	注册成功, 跳转到登录页面, 提示“注册成功”	通过
		确认密码: 123456		
TC-REG-02	手机号格式错误	姓名: 张三		
		手机号: 138001380	注册失败, 提示“手机号格式不正确”	通过
		密码: 123456		
TC-REG-03	手机号已注册	确认密码: 123456		
		姓名: 张三		
		手机号: 13800138001 (已存在)	注册失败, 提示“该手机号已注册”	通过
TC-REG-04	密码长度不足	姓名: 李四		
		手机号: 13800138002	注册失败, 提示“密码长度不能少于 6 位”	通过
		密码: 123		
TC-REG-05	两次密码不一致	确认密码: 123		
		姓名: 王五		
		手机号: 13800138002	注册失败, 提示“两次密码输入不一致”	通过

6.1.2 用户登录功能测试

用户登录是系统的入口功能，需要验证身份认证的正确性和安全性。测试用例如表 6-2 所示。

表 6-2 用户登录功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果
TC-LOGIN-01	正常登录	手机号:	登录成功, 跳转	通过
		13800138001 密码: 123456	到首页, 生成 Token	
TC-LOGIN-02	手机号不存在	手机号:	登录失败, 提示	通过
		13800138999 密码: 123456	"用户不存在"	
TC-LOGIN-03	密码错误	手机号:	登录失败, 提示	通过
		13800138001 密码: 错误密码	"密码错误"	
TC-LOGIN-04	账户被禁用	手机号:	登录失败, 提示	通过
		13800138003(已 禁用) 密码: 123456	"账户已被禁 用, 请联系管理 员"	
TC-LOGIN-05	空输入	手机号: 空	登录失败, 提示	通过
		密码: 空	"请输入手机号 和密码"	

6.1.3 服务预约功能测试

服务预约是系统的核心功能之一，需要验证预约流程的完整性和数据的正确性。测试用例如表 6-3 所示。

表 6-3 服务预约功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果
TC-BOOK-01	正常预约	服务：基础洗车	预约成功，生成订单号，跳 转到支付页面	通过
		日期：2025-11-15		
		时间：09:00-10:00		
		车牌号：粤 B12345		
		车型：丰田卡罗拉		
TC-BOOK-02	时间段已满	电话：13800138001	预约失败，提示“该时间段 已满，请选择其他时间”	通过
		服务：基础洗车		
		日期：2025-11-15		
		时间：10:00-11:00 (已满)		
		车牌号：粤 B12345		
TC-BOOK-03	车牌号格式错误	车型：丰田卡罗拉	预约失败，提示“车牌号格 式不正确”	通过
		电话：13800138001		
		服务：基础洗车		
		日期：2025-11-15		
		时间：09:00-10:00		
TC-BOOK-04	未登录预约	车牌号：ABC123 (格 式错误)	跳转到登录页面，提示“请 先登录”	通过
		车型：丰田卡罗拉		
		电话：13800138001		
		用户未登录状态下 提交预约		

续表 6-3 服务预约功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果
TC-BOOK-05	选择过期日期	服务：基础洗日期：		
		2025-11-01(已过期)		
		时间：09:00-10:00	预约失败，提示“不能选择	通过
		车牌号：粤 B12345	过期日期”	
		车型：丰田卡罗拉		
		电话：13800138001		

6.1.4 订单管理功能测试

订单管理功能允许用户查看、取消和评价订单。测试用例如表 6-4 所示。

表 6-4 订单管理功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果	用例编号
TC-ORDER-01	查看订单列表	用户 ID: 2	显示该用户的所有订单，包括订单号、服务名称、状态等信息	通过	TC-ORDER-01
TC-ORDER-02	查看订单详情	订单 ID: 1	显示订单的完整信息，包括服务详情、预约时间、车辆信息、支付状态等	通过	TC-ORDER-02
TC-ORDER-03	取消待确认订单	订单 ID: 2 (状态: 待确认) 取消原因: 时间冲突	订单状态更新为“已取消”，时间段预约数减 1，提示“取消成功”	通过	TC-ORDER-03

续 6-4 订单管理功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果	用例编号
TC-ORDER-04	取消已完成订单	订单 ID: 3			TC-ORDER-04
		(状态: 已	取消失败, 提示"订单	通过	
		完成)	已完成, 无法取消"		
		取消原因:			
		不满意			

6.1.5 支付功能测试

支付功能是系统的重要功能, 需要验证支付流程的安全性和准确性。测试用例如表 6-5 所示。

表 6-5 支付功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果
TC-PAY-01	支付宝支付成功	订单号: CW20251115001	跳转到支付宝支付页	通过
		支付方式: 支付宝 金额: 50 元	面, 支付成功后订单 状态更新为"已支付"	
TC-PAY-02	微信支付成功	订单号: CW20251115002	显示微信支付二维	通过
		支付方式: 微信支付 金额: 80 元	码, 扫码支付成功后 订单状态更新为"已 支付"	
TC-PAY-03	信用卡支付成功	订单号: CW20251115003		通过
		支付方式: 信用卡		
		卡号: 6222020200001234567	支付成功, 订单状态 更新为"已支付", 生	
		有效期: 12/26 CVV: 123	成支付记录	

续表 6-5 支付功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果
TC-PAY-04	重复支付	订单号: CW20251115004 (已支付)	支付失败, 提示“订单已支付, 无需重复支付”	通过
		支付方式: 支付宝		
TC-PAY-05	支付金额错误	订单号: CW20251115005	支付失败, 提示“支付金额与订单金额不符”	通过
		支付金额: 100 元 (订单金额 50 元)		

6.1.6 管理员功能测试

管理员功能包括预约管理、服务管理、用户管理等。测试用例如表 6-6 所示。

表 6-6 管理员功能测试用例

用例编号	测试项目	输入数据	预期结果	测试结果
TC-ADMIN-01	确认预约订单	订单 ID: 10 (状态: 待确认) 操作: 确认 服务名称: 豪华洗车	订单状态更新为“已确认”, 发送确认通知给用户	通过
TC-ADMIN-02	添加服务项目	价格: 150 元 时长: 60 分钟 描述: 包含内外清洗、打蜡	服务添加成功, 服务列表中显示新服务	通过
TC-ADMIN-03	编辑服务项目	服务 ID: 2 修改价格: 100 元→120 元	服务价格更新成功, 新订单使用新价格	通过
TC-ADMIN-04	删除服务项目	服务 ID: 5 (无关联订单)	服务删除成功, 服务列表中不再显示	通过
TC-ADMIN-05	禁用用户账户	用户 ID: 10 操作: 禁用	用户状态更新为“禁用”, 该用户无法登录系统	通过

6.2 性能测试

性能测试主要验证系统在高负载情况下的响应时间、并发处理能力和资源占用情况。本节使用 Apache JMeter 工具进行性能测试。

6.2.1 响应时间测试

响应时间是衡量系统性能的重要指标，测试结果如表 6-7 所示。

表 6-7 系统响应时间测试结果

测试接口	并发用户数	平均响应时间 (ms)	最大响应时(ms)	最小响应时(ms)	测试结果
用户登录	50	245	582	126	通过
服务列表查询	50	156	342	89	通过
创建预约订单	50	512	1023	287	通过
订单列表查询	50	278	645	145	通过
支付接口调用	50	823	1456	512	通过

测试结果表明，系统在 50 个并发用户的情况下，各接口的平均响应时间均在 1 秒以内（支付接口除外，因涉及第三方接口调用），最大响应时间控制在 2 秒以内，满足系统性能需求。

6.2.2 并发处理能力测试

并发处理能力测试验证系统在高并发情况下的稳定性，测试结果如表 6-8 所示。

表 6-8 系统并发处理能力测试结果

并发用户数	请求总数	成功请求数	失败请求数	成功率 (%)	平均响应时间 (ms)	测试结果
10	1000	1000	0	100	156	通过
50	5000	5000	0	100	312	通过
100	10000	9987	13	99.87	645	通过
200	20000	19854	146	99.27	1234	通过
500	50000	48723	1277	97.45	2856	基本通过

测试结果表明，系统在 100 并发用户以内时表现稳定，成功率接近 100%；在 200 并发用户时成功率仍在 99%以上；在 500 并发用户的高压力情况下，系统仍能维持 97%以上的成功率，基本满足系统设计要求 ‘。

6.3 兼容性测试

兼容性测试验证系统在不同浏览器和操作系统下的兼容性，测试结果如表 6-9 所示。

表 6-9 系统兼容性测试结果

浏览器	版本	操作系统	页面显示	功能正常	响应速度	测试结果
Google Chrome	120.0	Windows 11	正常	正常	快速	通过
Microsoft Edge	120.0	Windows 11	正常	正常	快速	通过
Firefox	121.0	Windows 11	正常	正常	快速	通过
Safari	17.0	macOS 14	正常	正常	快速	通过
Chrome	120.0	Android 13	正常	正常	一般	通过
Safari	17.0	iOS 17	正常	正常	一般	通过

测试结果表明，系统在主流浏览器和操作系统上均能正常运行，页面显示正常，功能完整，具有良好的兼容性。

6.4 安全性测试

安全性测试验证系统的身份认证、权限控制、数据加密等安全机制，测试结果如表 6-10 所示。

表 6-10 系统安全性测试结果

测试项目	测试内容	测试方法	预期结果	测试结果
身份认证	Token 验证	使用无效 Token 访问接口	返回 401 错误，提示“未授权”	通过
权限控制	角色权限验证	普通用户访问管理员接口	返回 403 错误，提示“权限不足”	通过

续表 6-10 系统安全性测试结果

测试项目	测试内容	测试方法	预期结果	测试结果
SQL 注入	输入恶意 SQL 语句	在登录框输入：' OR '1'=1	输入被过滤， 登录失败	通过
XSS 攻击	输入脚本代码	在备注框输入： <script>alert('xss')</script>	脚本被转义， 不执行	通过
密码加密	密码存储方式	查看数据库中的密码字段	密码经过 BCrypt 加密， 无法还原	通过
HTTPS 传 输	数据传输加密	使用 Wireshark 抓包分析	支付数据使用 HTTPS 加密传 输	通过
SQL 注入	输入恶意 SQL 语句	在登录框输入：' OR '1'=1	输入被过滤， 登录失败	通过

结论

本次课程论文设计从需求分析、系统设计、系统实现到系统测试，经历了一个完整的软件开发生命周期。在这个过程中，我不仅巩固了在校期间学习的理论知识，更重要的是将理论应用到实际项目中，积累了宝贵的实践经验。通过本次课程论文设计，我深刻体会到软件开发是一个系统工程，需要从用户需求出发，进行合理的架构设计和技术选型，编写高质量的代码，进行充分的测试验证，才能开发出一个真正可用、好用的系统。同时，我也认识到自己在技术能力、项目管理、问题解决等方面还存在不足，需要在今后的学习和工作中不断提升。最后，感谢导师在论文期间给予的悉心指导和帮助，感谢同学们的支持和鼓励。在今后的工作中,我将继续努力学习,不断提升自己的专业能力,为计算机软件行业的发展贡献自己的力量。

致谢

本论文的顺利完成，衷心感谢吴世旭老师的悉心指导与无私帮助。在整个课程项目研究与论文撰写过程中，吴老师以严谨的治学态度、丰富的实践经验和敏锐的学术洞察力，为本研究提供了全方位的宝贵指导。从课题选择、技术路线确定、系统架构设计到具体实现细节，吴老师都给予了耐心细致的指导，使我对企业级应用系统开发有了更为深入的理解和实践体验。

吴老师不仅在专业技术层面给予我充分指导，更在工程思维培养、系统性问题分析和解决方案设计等方面对我进行了系统训练。他始终鼓励我独立思考、勇于探索实践，这种培养方式对我今后的学术研究和职业发展都将产生深远影响。在项目遇到技术难题时，吴老师总能一针见血地指出问题关键，并提供切实可行的解决方案，使项目得以顺利推进。

特别感谢吴老师在系统架构设计方面给予的关键性指导。在基于 SpringBoot 的后端架构设计和 Vue.js 前端框架选型过程中，吴老师以其丰富的项目经验，帮助我们确定了最合适的技术方案。在数据库设计、接口规范定义等关键环节，吴老师的建议使系统架构更加合理规范。此外，在支付接口集成、安全机制实现等关键技术难点上，吴老师的指导让我们少走了许多弯路。在此，谨向吴世旭老师致以最诚挚的感谢和崇高的敬意！

同时，也感谢在项目开发过程中给予我帮助和支持的同学们，以及所有关心和支持我的人。

最后，感谢老师在百忙之中审阅本课程论文并提出宝贵意见。

参考文献

- [1] 程千蛟.关税战背景下汽车后市场发展面临的机遇和挑战[J].汽车维修与修理,2025,(09):6-11.DOI:10.16613/j.cnki.1006-6489.2025.09.001.
- [2] 迟梦园,罗朝军,董丽薇.基于 O2O 汽车清洗营销战略的分析与研究[J].现代商业,2017,(33):21-22.DOI:10.14097/j.cnki.5392/2017.33.007.
- [3] How digitalization and information technology adoption affect firms' innovation performance: evidence from Chinese automotive firms[J].European Journal of Innovation Management,2025,28(6):2512-2531.
- [4] 赵燕.基于 Dubbo 的汽车养护预约系统的设计与实现[D].西安电子科技大学,2020.DOI:10.27389/d.cnki.gxadu.2020.001960.
- [5] 苏小伟.基于 SpringBoot 的汽车维修企业智能化管理系统的设计与实现[D].北京交通大学,2022.DOI:10.26944/d.cnki.gbfju.2022.000750.
- [6] 鄂雪妮,沈志涛,王超.基于 Springboot 微服务架构的移动网络用户投诉预处理系统设计与实现[J].长江信息通信,2025,38(01):115-117.DOI:10.20153/j.issn.2096-9759.2025.01.033.
- [7] 颜光,刘梦雅,李文美.O2O 模式下移动洗车服务市场前景的探究[J].现代经济信息,2016,(08):350.
- [8] 李文峰,冯永明,袁海润.基于微信平台的洗车服务系统[J].西安科技大学学报,2017,37(03):445-449.DOI:10.13800/j.cnki.xakjdxxb.2017.0321.
- [9] XIONG X ,MEI Q .Analysis on the Business Mode of Sports and Fitness O2O Platform in the Context of “Internet plus Sports”[C]//Science and Engineering Research Center.Proceedings of 2016 International Conference on Education, Training and Management Innovation(ETMI 2016).Zhuhai College of Jilin University;School of Business, Macau University of Science and Technology;,2016:229-236.
- [10] 李唯.基于 SpringBoot+Mybatis 的驾校预约系统设计与实现[J].电脑编程技巧与维护,2022,(03):10-12.DOI:10.16184/j.cnki.comprg.2022.03.003.
- [11] 王佚尊.O2O 平台用户知识隐藏行为形成机理研究[D].西安理工大学,2024.DOI:10.27398/d.cnki.gxalu.2024.001511.

- [12]王婧涵.途虎持续提升汽车后市场品牌影响力[N].中国证券报,2025-08-29(A06).DOI:10.28162/n.cnki.nczjb.2025.003467.
- [13]Elizaveta K .Optimization of Controlled Queueing Systems: the Case of Car Wash Services[J].Transportation Research Procedia,2021,54662-671.
- [14]刘辉.DeepSeek 赋能的自适应异构计算平台：架构、智能调度与运维优化[J].电脑知识与技术,2025,21(22):1-4.DOI:10.14004/j.cnki.ckt.2025.1142.
- [15]陈伶.网上服务预约模块的设计与实现[J].计算机与数字工程,2010,38(01):201-204.DOI:CNKI:SUN:JSSG.0.2010-01-055.
- [16]董袁泉,贾苏,钱梦颖.基于 SpringBoot 的自动化车座安排系统[J].电脑知识与技术,2023,19(02):47-49.DOI:10.14004/j.cnki.ckt.2023.0058.
- [17]曹思琳.基于 Websocket 服务器的汽车售后服务预约系统设计[J].微型电脑应用,2021,37(06):38-41.
- [18]曾青松,魏斌.基于 RESTful API 的访问权限系统的设计与实现[J].电脑编程技巧与维护,2020,(11):3-6.DOI:10.16184/j.cnki.comprg.2020.11.001.
- [19]胡传甫,王昕葳,周宸,等.业务逻辑漏洞常见类型分析与 JWT 测试示例研究[J].计算机时代,2025,(02):6-10.DOI:10.16644/j.cnki.cn33-1094/tp.2025.02.002.
- [20]李宗辉.基于响应式设计的移动端 UI 动态适配框架研究[C],2025:653-656.DOI:10.26914/c.cnkihy.2025.013234.